Approximating Functions: A Practical Guide to Sollya and Function Implementation

CCMATH

1 How to use this guide

The intended reader is a C++ programmer with enough calculus to recognize a derivative and a short series expansion. The guide starts one level above the usual libm implementation details. A call to `exp`, `log`, or `sin` is reduced, approximated on a small interval, reconstructed, and rounded, with generated constants and checks along the way. The examples avoid CCMath-specific layout so the same habits still apply when reading another math library.

Treat the printed numbers as artifacts, not prose. The Sollya scripts, C++ oracle harness, and Gappa proof file used by the listings live under `tools/proofs/math/approx_doc`. Expected-output blocks come from those files. The bibliography uses ACM numeric style, and the table near the end records which sources support which kinds of claims.

Do not read it straight through the first time. Read Section 2 through Section 8 for vocabulary, and leave the scripts alone. Then install Sollya (Section 12.1) and run the small sine kernel before the table-based $\log_2$ core. Reduction, tables, precision tiers, and rounding make more sense after at least one polynomial has been generated and checked on your machine.

Figure 1 is the map for the rest of the guide. It uses $\exp(x) = 2^k \exp(r)$ for one concrete input. Reduction creates the small residual r, the polynomial is written only for that residual, and reconstruction restores the scale.

2 The core problem

A handful of operations are *correctly rounded* on every conforming platform: addition, subtraction, multiplication, division, square root, and fused multiply-add are required by IEEE 754 to behave as if the exact result were computed to infinite precision and rounded once [14]. Almost everything else cannot be handled that way.

`exp`, `log`, `sin`, `cos`, `atan`, `pow`, `tgamma`, and the rest are transcendental or algebraic-but-irrational. For almost every representable input the exact value is irrational, so no finite sequence of additions, multiplications, divisions, and square roots produces it. An implementation can only compute a value close enough that it rounds to the right answer in the target format.

Implementing such a function is an engineering process with four recurring stages [17, 18]:

(1) **Argument reduction.** Use mathematical identities to map the input into a small, well-behaved interval.
(2) **Approximation.** Replace the true function by a polynomial whose error is small and bounded by a proof.
(3) **Reconstruction.** Combine the approximation with information discarded during reduction to recover the full-range result.
(4) **Rounding.** Deliver the result correctly (or faithfully) rounded in the target format, in every rounding mode the function supports.

Even the familiar series do not make the direct approach viable. The Taylor series for sine converges for every real input, but a naive double-precision sum is not a usable implementation. For $x = 100$, the largest term of the series is about $1.1 \cdot 10^{42}$, while the final result lies in $[-1, 1]$. The leading 42 digits must cancel, and a double carries about 16. Every useful digit is destroyed before the sum settles. Reduction keeps the polynomial on a small interval where nothing like this can happen.

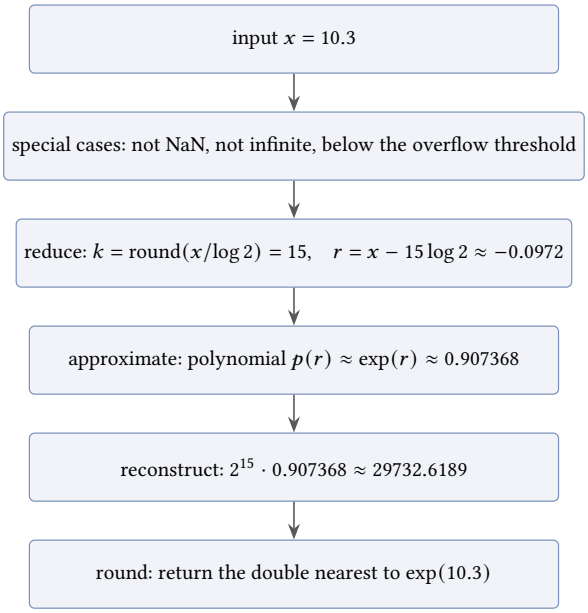

Fig. 1. The four-stage pipeline on a concrete call. The reduction moves the power-of-two scale into the integer k, the polynomial only ever sees the small residual r, and the reconstruction puts the scale back.

Input	Polynomial alone	Reduce, then polynomial
$x = 1$	$7.6 \cdot 10^{-13}$	below $6.7 \cdot 10^{-10}$
$x = 10$	$5.5 \cdot 10^{2}$	below $6.7 \cdot 10^{-10}$
$x = 100$	$1.6 \cdot 10^{16}$	below $6.7 \cdot 10^{-10}$

A full implementation is not needed to see the problem. Take the degree 13 Taylor polynomial for sine and use it two ways: directly on the raw input, and as the kernel behind a reduction modulo $\pi/2$.

The right column assumes an accurate reducer and quotes the certified bound of the same polynomial on $[-\pi/2, \pi/2]$. The left column shows the cost of skipping reduction: error around 550 for $x = 10$, while the true result never leaves $[-1, 1]$.

Stages 1, 3, and 4 are function-specific algebra and careful floating-point bookkeeping. Stage 2 is where Sollya is used.

3 Polynomial approximation

Once reduction has made the interval small, the usual replacement is a polynomial. It uses operations the hardware already provides, fits Horner or FMA-heavy evaluation schemes, and gives analysis tools something finite to bound.

Writing down a polynomial is not the hard part. The hard part is choosing coefficients that spend the available degree on the whole interval, under the same error model that the final implementation will claim.

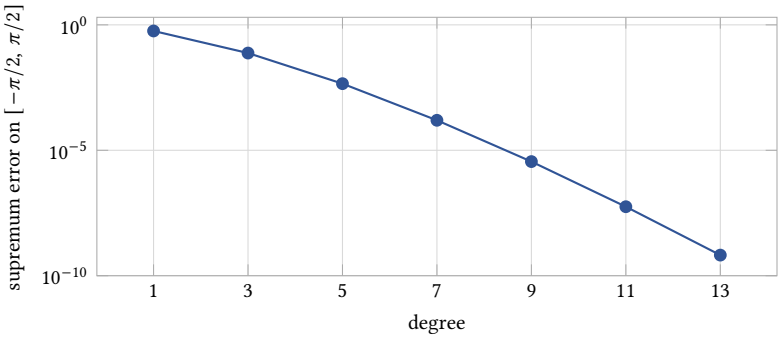

Fig. 2. Supremum absolute error of odd Taylor sine polynomials on $[-\pi/2, \pi/2]$ as a function of degree, certified with Sollya's `supnorm`. Adding terms reduces the error, but the distribution is worst-case at the interval endpoints.

4 Taylor approximation

A Taylor series centered at zero has the form

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(0)}{k!} x^k.$$

Truncating at degree n gives a polynomial whose error is smallest at $x = 0$ and grows toward the interval edges.

That behavior comes from what the Taylor polynomial is allowed to know. It uses the value of f and the first n derivatives at the center, and it has no direct information about the rest of the interval. The fit is excellent near the center and has nothing holding it down near the endpoints, which is where the worst-case error accumulates.

Small spot checks are misleading here. A Taylor polynomial can pass the inputs near its center and still lose badly at the endpoints, so the interval supremum is the quantity that drives the design.

Once the degree and storage formats are fixed, the usual next tool is minimax approximation.

5 Minimax approximation

A minimax polynomial is chosen by the worst input on the interval: it minimizes the largest error, not the average error and not the error near one favored point.

That matches the usual libm objective. The implementation must answer any input in the reduced interval, so a polynomial that spends all its accuracy at the center is usually the wrong object.

Minimax comes in flavors. The error being minimized can be absolute, relative, or weighted by any positive function, and the search can be constrained, for example to odd powers only or with the leading coefficient pinned to an exact value. Production kernels almost always use the relative or weighted form, for ULP reasons covered in Section 8, and the constrained forms appear whenever symmetry or an exact leading term is part of the design.

One useful mental model is a fixed budget. A degree n polynomial has $n + 1$ coefficients. Lowering the error in one part of the interval usually raises it somewhere else. At the optimum, the remaining peaks have been balanced against each other: they reach the same magnitude and alternate in sign. This is the equioscillation pattern.

For a degree n polynomial, the best minimax approximation has an error curve that reaches its maximum size at least $n + 2$ times, with alternating signs [17].

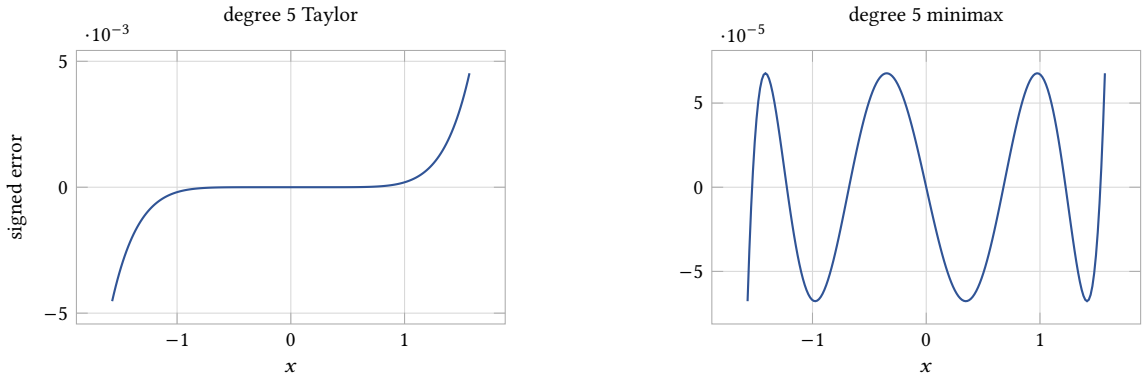

Fig. 3. Signed error of two degree 5 approximations to $\sin(x)$ on $[-\pi/2, \pi/2]$. Note the scales: the Taylor peak is about 67 times taller, even though both polynomials have the same degree and cost the same to evaluate. The minimax curve equioscillates, touching its bound $n + 2 = 7$ times with alternating signs.

Use the theorem during review. One dominant spike with long quiet regions is a warning sign. Balanced alternating peaks do not prove the whole implementation, but they are evidence that the approximation is using its degree for the intended interval.

Taylor and minimax spend the same degree in very different places. Figure 3 shows the difference directly. Both polynomials have degree 5 and cover the same interval. The Taylor error climbs monotonically to $4.5 \cdot 10^{-3}$ at the endpoints. The minimax error never exceeds $6.8 \cdot 10^{-5}$, about 67 times smaller, and shows the seven equal alternating peaks the theorem predicts.

6 Remez

Remez exchange is the workhorse algorithm behind most of the minimax polynomials used in this style of implementation.

Remez works backward from the equioscillation property. It guesses where the $n + 2$ worst-error points should sit, then solves a linear system for the unique polynomial whose error at exactly those points has equal magnitude and alternating sign. That polynomial is optimal for the guessed points, but its true error peaks may lie elsewhere. The algorithm then finds the actual extrema of the error curve, moves the guessed points there, and repeats. When the true peaks stop moving, the error curve has the minimax shape and the loop stops [17]. Convergence is usually fast once the reference points sit near their final positions, though hard functions and poor starting points can make the early iterations wander.

In Sollya this exploratory step is the `remez` command.

Remez is still one abstraction too high for source code. Its coefficients are real numbers. The implementation has `float`, `double`, `long double`, fixed-width constants, or explicit expansions such as double-double pairs.

Use the real-coefficient result to choose the shape and degree. Do not treat it as the final source polynomial.

7 Coefficients must fit the machine

Hand-rounding a real-coefficient minimax polynomial is a common way to lose the property that made the polynomial useful. The rounded version can have a different error shape, and the bound can grow enough to miss the target [17].

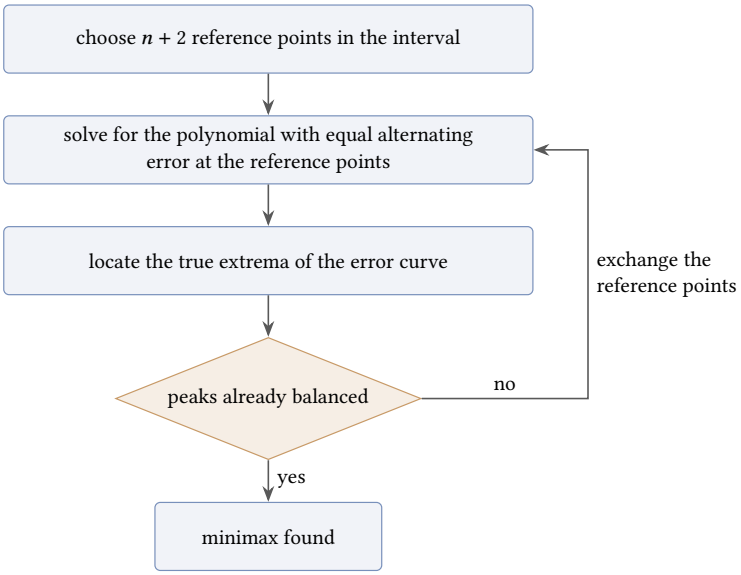

Fig. 4. The Remez exchange loop. Each pass pins the error at the current reference points, then checks whether the real peaks agree.

Step	Use
`remez`	Explore the real-coefficient approximation
`fpminimax`	Generate coefficients for source code

Minimax optima are sensitive. The search tuned every coefficient against every other coefficient to make the peaks equal. Rounding each coefficient independently perturbs the error curve by amounts comparable to the gap the optimization fought for, and the perturbations do not cancel reliably. Finding the best machine-representable polynomial is its own optimization problem, separate from rounding the real-coefficient one.

Sollya's `fpminimax` command addresses this problem.

`fpminimax` searches for a near-minimax polynomial whose coefficients are already constrained to the requested formats [3, 18].

In practice, keep the split simple:

Pitfall: constants that are almost right

Do not hand-round production coefficients unless the rounded polynomial is checked again. A bad constant can ruin a good algorithm. A decimal literal that looks harmless may round differently than the value Sollya produced. Prefer hexadecimal constants when the value is meant to be exact in the target format.

8 Absolute error and relative error

Pick the error model before generating coefficients.

Absolute error is:

$$|p(x) - f(x)|$$

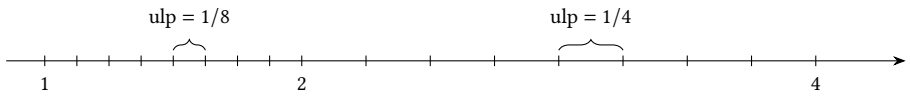

Fig. 5. Representable values of a toy binary format with three significand bits. The spacing is constant inside each binade and doubles at every power of two. Real formats behave the same way with much finer spacing.

Relative error is:

$$\left|\frac{p(x)}{f(x)} - 1\right|$$

ULP-based accuracy pushes the choice toward relative error. ULP size changes with the magnitude of the result [18], so the same absolute error can be harmless in one binade and disastrous in another.

Figure 5 shows why. Floating-point values are not evenly spaced. Within one binade, the interval between consecutive powers of two, the spacing is constant. At each power of two it doubles. A fixed absolute error therefore costs a different number of ULPs depending on where the result lands: in double precision, an error of 2^{-55} is an eighth of an ULP for a result near 1 and 128 ULPs for a result near 2^{-10}. Relative error divides that scale out. A relative bound translates into a roughly uniform ULP bound at every magnitude, which is why it is the usual target when the function's result spans many binades.

Relative error still has traps. Near a zero of f, the division by $f(x)$ can dominate the measurement instead of describing the implementation risk. In that case, change the kernel. Section 11 lists the usual transforms.

Two more regions deserve care. When the result is subnormal, the format quantizes more coarsely than the relative model assumes, so relative error and ULP distance part ways there. When the result sits near the overflow threshold, a relative polynomial bound says nothing about whether the rounded result becomes infinity. Both regions belong to the special-case plan, not to the polynomial.

9 Sollya

Use Sollya for the approximation work in this guide: coefficient generation, quick exploration, and certified interval checks [4].

In this guide, Sollya is used for the following jobs:

9.1 Running Sollya

A script run looks like this:

```
sollya script.sollya
```

Start scripts like this:

```
prec = 200!;
display = hexadecimal!;
```

The precision setting controls Sollya's working precision. It should be higher than the target format. For double coefficients, 200 bits is a reasonable starting point.

Table 1. Sollya commands used in this guide.

Task	Sollya command
Estimate the needed degree	`guessdegree`
Compute a real-coefficient minimax polynomial	`remez`
Compute machine-format coefficients	`fpminimax`
Quickly estimate worst-case error	`dirtyinfnorm`
Compute a safer error bound	`infnorm` or `supnorm`
Print constants for source code	`display = hexadecimal`
Print Horner form	`horner`
Print expanded form	`canonical`

Print production constants in hexadecimal. That gives C++ the same floating-point value Sollya printed. A decimal spelling can add a second rounding step.

10 A small Sollya example

For a first Sollya run, start after range reduction. Assume the input has already been reduced from $\exp(x)$ to $\exp(r)$ with r close to zero.

The usual shape is:

$$\exp(x) = 2^k \exp(r)$$

where r lies in an interval such as:

$$\left[-\frac{\log(2)}{2}, \frac{\log(2)}{2}\right]$$

The following script is intentionally small enough to run as printed:

```
prec = 200!;

f = exp(x);
I = [-log(2) / 2; log(2) / 2];

print("suggested degree for 1b-53 relative:");
guessdegree(f, I, 1b-53);

p = fpminimax(f, 11, [|D, D, D, D, D, D, D, D, D, D, D, D|], I, relative);

print("coefficients:");
display = hexadecimal!;
for i from 0 to 11 do coeff(p, i);
display = decimal!;

eps_fast = dirtyinfnorm(p / f - 1, I);
print("quick relative error estimate:", eps_fast);

eps_cert = supnorm(p, f, I, relative, 1b-40);
print("certified relative error:", eps_cert);
```

The format list `[|D, D, . . . |]` names the storage format of each coefficient from degree 0 upward, one entry per coefficient, D meaning double. Mixed lists are common in real kernels, for example a double-double pair for the leading coefficients and plain doubles for the tail.

Expected output, with the long decimals shortened:

```
suggested degree for 1b-53 relative:
[11;11]
coefficients:
0x1p0
0x1p0
0x1.000000000000bp-1
0x1.5555555555511p-3
0x1.55555555502a1p-5
0x1.1111111122322p-7
0x1.6c16c1852b7bp-10
0x1.a01a014761f6ep-13
0x1.a01997c89e6bp-16
0x1.71dee623fde67p-19
0x1.28af3fca7ab0cp-22
0x1.ade156a5dbe77p-26
quick relative error estimate: 4.0961995708e-18
certified relative error: [4.0961995708e-18; 4.0961995712e-18]
```

Read the output as a set of design checks. `guessdegree` reports that degree 11 is the first degree that can reach 2^{-53} on this interval, so the script asks for that degree rather than guessing. The first coefficients sit on the Taylor values $1, 1, 1/2, 1/6$ to within a few ULPs, which is a good sanity check. The certified bound is an enclosure, not a point value: the true supremum lies inside the printed interval, so quote its upper end, $4.1 \cdot 10^{-18} \approx 2^{-57.8}$. Read $\log_2(\text{error})$ as a count of correct bits. This polynomial has margin past double precision, which the evaluation and reduction stages will spend.

A packaged copy of this script ships as `tools/proofs/math/approx_doc/sollya/exp_small_fpminimax.sollya`.

That bound covers only the exact polynomial approximation to $\exp(r)$ on that interval. A complete `exp` still has to prove the reducer, rounded evaluation, reconstruction, special cases, and final rounding.

11 Choosing the kernel

After reduction, the implementation usually works with a smaller kernel rather than the public function directly.

For $\sin(x)$, the kernel may be $\sin(r)$.

For small x, it may be better to approximate $\sin(r)$ divided by r.

For `log1p`, it may be better to approximate $\log(1 + r)$ divided by r.

For $\text{expm1}(x)$, it may be better to approximate $\text{expm1}(r)$ divided by r.

A useful kernel removes cancellation, keeps the relative error model stable near zero, and can reduce the polynomial degree.

Do the algebra before asking Sollya for coefficients.

What should be small after reduction, and what should be exact? That choice often determines the kernel.

11.1 Small-input transforms

$$\log(1+x) = x \cdot \frac{\log(1+x)}{x}$$

$$\exp(x) - 1 = x \cdot \frac{\exp(x) - 1}{x}$$

$$\sin(x) = x \cdot \frac{\sin(x)}{x}$$

$$\cos(x) = 1 - 2\sin^2\left(\tfrac{x}{2}\right)$$

These are mainly small-input tools. By factoring out the term that causes cancellation or unstable relative error, they leave a smoother quotient or a reconstruction identity that preserves more useful bits.

The quotient forms rely on removable singularities. $\sin(x)/x$ is undefined at $x = 0$, but its limit there is 1, and the function extended with that value is smooth across the whole interval. That extended function is what the polynomial approximates, and Sollya handles the extension automatically. The implementation never divides. It evaluates the polynomial and multiplies by x, so the input $x = 0$ flows through as $x \cdot p(0) = 0$ with the correct sign.

The identity must still be checked on the target interval and under the target rounding policy. None of these forms is automatically best on every interval.

12 Worked example: a small sine kernel

Start here for hands-on work

If you want to run Sollya and follow code in order, begin with Section 12.1 and Section 12.2. Skim Section 2 through Section 8 first if the approximation vocabulary is new.

The first kernel is deliberately small, so all of the moving parts stay visible:

$$\texttt{sin\_small}(x) \approx \sin(x) \quad \text{for } |x| \le \tfrac{1}{16}$$

It is not a public `sin`. There is no quadrant logic, no large-input reduction, and no special-case table. The restriction keeps the kernel small enough to audit on one screen.

12.1 Set up Sollya

Only Sollya is required for the worked examples. Install it once, then run the companion scripts from the repository root:

```
# macOS
brew install sollya

# Debian/Ubuntu
sudo apt install sollya
```

```
sollya tools/proofs/math/approx_doc/sollya/sin_small_error.sollya
```

The expected output after each script was produced with Sollya 8.0. Numbers certified by `supnorm` are reproducible across versions, although incidental formatting may differ. The build script reruns the same scripts when the guide PDF is rebuilt. MPFR, Gappa, Rocq, and SageMath are useful later for validation, but they are not required to follow the examples here.

12.2 Specify the target

Target:	$f(x) = \sin(x)$
Input interval:	$x \in \left[-\frac{1}{16}, \frac{1}{16}\right]$
Output type:	`double`
Accuracy target for this example:	well below 1 ULP on the stated interval under round-to-nearest

12.3 Choose the kernel

Directly approximating $\sin(x)$ works on this interval, but the odd symmetry is useful. Since $\sin(x)$ is odd:

$$\sin(x) = xq(x^2)$$

where:

$$q(z) = \frac{\sin(\sqrt{z})}{\sqrt{z}}$$

Near zero, $q(z)$ is smooth and close to 1.

The implementation will compute:

$$z = x^2$$

and:

$$\sin(x) \approx x\,p(z)$$

where p approximates q.

12.4 Pick a simple polynomial

For the teaching version, Taylor coefficients make the arithmetic easy to recognize:

$$\sin(x) = x - \frac{x^3}{6} + \frac{x^5}{120} - \frac{x^7}{5040} + \frac{x^9}{362880} + \cdots$$

Factoring out x gives:

$$\sin(x) = x\left(1 - \frac{z}{6} + \frac{z^2}{120} - \frac{z^3}{5040} + \frac{z^4}{362880} + \cdots\right)$$

where $z = x^2$.

Production code would use `fpminimax` coefficients here. Taylor values keep the arithmetic recognizable while the workflow is new, and Section 12.8 runs `fpminimax` on this same kernel so the two can be compared. Either way, Sollya checks the error of whatever polynomial actually ships.

12.5 Check the approximation in Sollya

Now bound the error on the exact interval the kernel promises:

```
prec = 200!;
display = hexadecimal!;

I = [-1/16; 1/16];

p = x * (
    0x1.0000000000000p+0
  + x^2 * (
      -0x1.5555555555555p-3
    + x^2 * (
        0x1.1111111111111p-7
      + x^2 * (
          -0x1.a01a01a01a01ap-13
        + x^2 * 0x1.71de3a556c734p-19
      )
    )
  )
);

abs_err = supnorm(p, sin(x), I, absolute, 1b-80);
rel_err = supnorm(p, sin(x), I, relative, 1b-80);

display = decimal!;
print("absolute error:", abs_err);
print("relative error:", rel_err);
```

Expected output, shortened:

```
absolute error: [3.6826560686e-21; 3.6826560686e-21]
relative error: [5.8960875588e-20; 5.8960875588e-20]
```

So the certified approximation error of this polynomial is $3.69 \cdot 10^{-21}$ absolute and $5.90 \cdot 10^{-20} \approx 2^{-63.9}$ relative on the stated interval. Both numbers reappear in the worked budget of Section 12.9.

The familiar series does not justify the code. The checked interval and the reported bound do.

12.6 Implement the kernel

Horner form translates directly into the C++ kernel:

```
[[nodiscard]] constexpr double sin_small(double x) noexcept
{
    const double z = x * x;
```

```
    const double p =
        (((0x1.71de3a556c734p-19 * z
         - 0x1.a01a01a01a01ap-13) * z
         + 0x1.1111111111111p-7) * z
         - 0x1.5555555555555p-3) * z
         + 0x1.0000000000000p+0;

    return x * p;
}
```

This kernel does not defend its interval. Its caller must supply values with $|x| \le 1/16$. In a real internal path, the surrounding reducer owns that precondition, and the tests should call the kernel only through reduced inputs or explicit reduced-interval cases.

The kernel is plain C++17. Every number quoted for it in this guide was measured with `-O2` and no fast-math flags. Section 20 explains why the flags are part of the claim.

12.7 Test the kernel

For this restricted kernel, useful tests include:

(1) x equals 0
(2) x equals the interval endpoints
(3) several values near the endpoints
(4) positive and negative inputs with matching signs
(5) comparison against MPFR or another high-precision oracle

For round-to-nearest tests, compare the returned bit pattern or ULP distance. Printed decimal text is the wrong target.

Example values:

$$0, \quad \frac{1}{1024}, \quad -\frac{1}{1024}, \quad \frac{1}{32}, \quad -\frac{1}{32}, \quad \frac{1}{16}, \quad -\frac{1}{16}$$

Keep the endpoints in the test set. The approximation was proved only on that interval. If a later change expands the interval to $[-1/8, 1/8]$, the proof and tests must be redone.

ULP distance is worth implementing once and reusing everywhere. The trick is to map doubles onto the integers so that consecutive finite doubles map to consecutive integers:

```
std::int64_t ulp_order(double v)
{
    std::int64_t bits;
    std::memcpy(&bits, &v, sizeof bits);
    return bits >= 0 ? bits : INT64_MIN - bits;
}

std::int64_t ulp_distance(double a, double b)
{
```

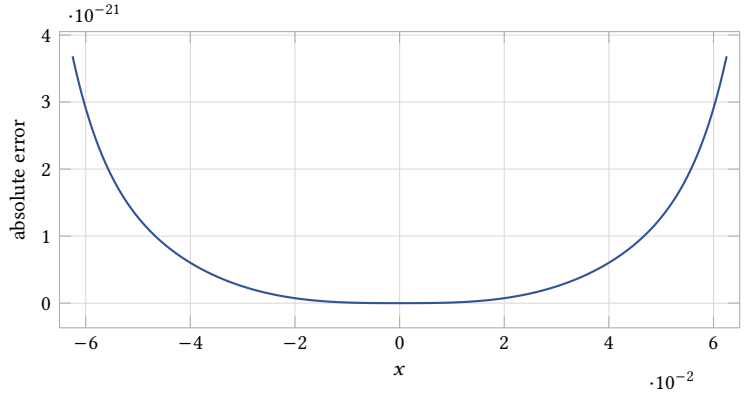

Fig. 6. Absolute error of the small sine teaching polynomial on $[-1/16, 1/16]$, sampled in Sollya at high working precision. Double arithmetic cannot even measure errors this small directly, which is why the curve comes from the script and not from a C++ harness.

```
    const std::int64_t d = ulp_order(a) - ulp_order(b);
    return d < 0 ? -d : d;
}
```

The ordering works because IEEE doubles with the same sign already compare like integers bit-for-bit. The negative branch folds the other half of the line into the same order. A distance of 0 means correctly rounded against the oracle, and a distance of 1 means the adjacent double, which is the faithful-rounding criterion.

A complete oracle harness using this, with MPFR as the reference, ships as `tools/proofs/math/approx_doc/cpp/sin_small_check.cpp`. It sweeps the listed special values plus a dense grid. Its output on this kernel:

```
inputs tested:      200001
correctly rounded:  162160
max ULP distance:   1
worst input:        -0x1.fffeb074a771dp-5
```

On that grid, the kernel is faithful everywhere and correctly rounded for 81 percent of the tested inputs. Its worst input is effectively an endpoint case, which is where this error curve is largest. A useful status table records those measurements alongside the proof bounds; the polynomial bound alone is not enough.

12.8 A production polynomial for the same kernel

The Taylor coefficients were only for inspection. For the same degree, interval, and storage format, `fpminimax` produces the following coefficients, from `tools/proofs/math/approx_doc/sollya/sin_small_fpminimax.sollya`. A direct request fails here: asking `fpminimax` for an odd polynomial against sin with relative error degenerates, because the odd basis is not a Chebyshev system for that weighted problem. The usual workaround is to fit the transformed kernel $\sin(x)/x$ with even monomials instead, then multiply by x in the implementation:

```
I = [0; 1/16];
```

```
Icheck = [-1/16; 1/16];

q = fpminimax(sin(x)/x, [|0, 2, 4, 6, 8|], [|D, D, D, D, D|], I, relative);
p = x * q;
```

Expected output:

```
coefficients of q, so that sin(x) ~ x * q(x^2):
  q0 = 0x1p0
  q2 = -0x1.5555555555555p-3
  q4 = 0x1.111111110f491p-7
  q6 = -0x1.a01a00e16af3ep-13
  q8 = 0x1.71c24233f1bafp-19
certified absolute error: [5.1436720042e-23; 5.1436720042e-23]
certified relative error: [2.0923182501e-21; 2.0923182501e-21]
```

Compare against the Taylor bounds from the check script. The degree and the evaluation cost have not changed, and the certified absolute error drops from $3.7 \cdot 10^{-21}$ to $5.1 \cdot 10^{-23}$, about 70 times. The first two coefficients match the Taylor values to the last bit. The lower ones drift away from them, because the optimizer adjusts the tail coefficients that the Taylor series ties to derivatives at zero.

12.9 Proving the evaluation error

Sollya bounded the distance from the exact polynomial to sin. The compiled kernel evaluates that polynomial in floating point, and every operation rounds. Gappa bounds this second distance. The proof file ships as `tools/proofs/math/approx_doc/gappa/sin_small_eval.gappa` and its core mirrors the C++ operation for operation:

```
x = rnd(x_);

z rnd= x * x;
p rnd= (((c9 * z + c7) * z + c5) * z + c3) * z + c1;
s rnd= x * p;

Mz = x * x;
Mp = (((c9 * Mz + c7) * Mz + c5) * Mz + c3) * Mz + c1;
Ms = x * Mp;

{ |x| <= 0x1p-4 -> |s - Ms| <= 0x1.02p-57 /\
  x in [0x1p-60, 0x1p-4] -> |s -/ Ms| <= 1b-52 }
```

Running `gappa` on the file succeeds silently, which is how Gappa reports a proof. The two goals say: the computed value s sits within $2^{-56.99}$ of the exact polynomial value absolutely, and within 2^{-52} relatively on the positive half. The kernel is odd under exact negation, so the relative bound covers the negative half too.

Obligation	Tool	Absolute	Relative
approximation, p vs sin	Sollya `supnorm`	$3.7 \cdot 10^{-21}$	$5.9 \cdot 10^{-20}$
evaluation, s vs p	Gappa	$7.0 \cdot 10^{-18}$	2^{-52}
oracle agreement	MPFR harness	max 1 ULP over 200001 inputs	

Now the budget for this kernel can be filled in with proven numbers instead of placeholders:

The budget and the harness hit the same limit. Near the top of the interval, $\mathrm{ulp}(\sin x) = 2^{-57}$, and the proven evaluation error nearly fills half of it, so correct rounding cannot be guaranteed there. The harness agrees: faithful everywhere on the grid, correctly rounded only 81 percent of the time. Tightening the proof to match the observed behavior would mean splitting the interval and proving scaled bounds piece by piece, which is where production correct-rounding proofs put most of their effort.

Milestone. You can now take a smooth function on a small interval, generate a polynomial, certify its approximation error, bound its evaluation error, and check the result against an oracle. That is the complete inner loop of kernel work.

12.10 What this example leaves out

A full implementation still has additional obligations:

(1) reduction for arbitrary finite inputs
(2) quadrant reconstruction
(3) handling of NaN and infinities
(4) signed-zero behavior
(5) directed rounding behavior
(6) large-input Payne-Hanek reduction
(7) validation against an oracle
(8) a recorded error budget for reduction, evaluation, and reconstruction

Larger functions use the same loop, then add reduction, tables, multiple precision tiers, special cases, and harder rounding decisions.

Checkpoint

Before the next example, check the kernel you just built.

(1) Why approximate $\sin(x)/x$ in $z = x^2$ instead of $\sin(x)$ directly?
(2) The interval grows from $[-1/16, 1/16]$ to $[-1/8, 1/8]$. What must be redone?
(3) Why are the coefficients written in hexadecimal?

Answers. The quotient is smooth and close to 1 near zero, so its relative error stays well behaved, and working in x^2 uses the odd symmetry to halve the number of terms. The Sollya bound and the tests hold only on the stated interval, so both must be rerun, and the degree may need to grow. Hexadecimal literals store exactly the bits Sollya produced, with no extra decimal rounding step.

13 Advanced worked example: a small log2 core

The next example adds the pieces missing from the small sine kernel: table reduction, a transformed kernel, split constants, and a polynomial on a small residual interval. A production-grade log with this structure, including its correct-rounding proof, is described in [9]. Read that paper after this section and the structure will look familiar.

It is still not a full $\log_2$ implementation. It is the core used after the input has already been decomposed as:

$$x = m2^e$$

with:

$$m \in [1, 2)$$

The full result is:

$$\log_2(x) = e + \log_2(m)$$

The core computes the $\log_2(m)$ part.

13.1 Choose the table reduction

Divide $[1, 2)$ into B buckets. For a teaching version, use:

$$B = 16$$

For each bucket, choose a value r_i close to the reciprocal of the bucket center. For an input mantissa m, compute:

$$u = mr_i - 1$$

If r_i is well chosen, u is small. The identity is:

$$m = \frac{1 + u}{r_i}$$

so:

$$\log_2(m) = \log_2(1 + u) - \log_2(r_i)$$

The table stores r_i and a split form of $-\log_2(r_i)$.

One split is:

$$-\log_2(r_i) = h_i + \ell_i$$

The high part h_i can be rounded to fewer bits so that adding it to the exponent is exact in the fast path. The low part ℓ_i carries the remaining bits.

13.2 What the table buys

The table exists to shrink the polynomial interval. With $B = 16$, a simple bucket-center choice leaves a residual roughly bounded by:

$$|u| \le \frac{1}{2B}$$

The exact bound depends on how the buckets and reciprocals are chosen. More table bits shrink the interval that the polynomial has to cover.

That is the trade: table entries cost memory and cache pressure, while a smaller residual can lower the degree or make the proof easier.

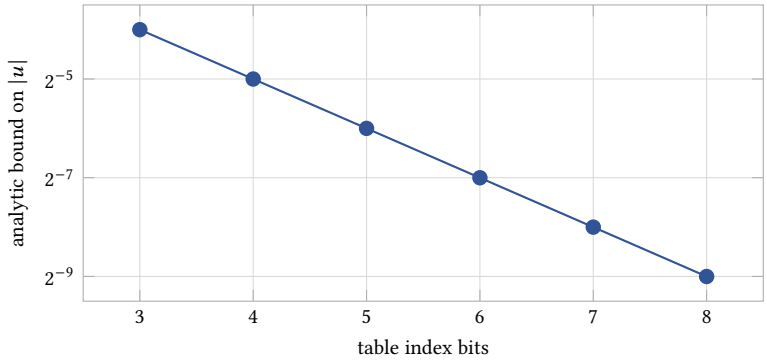

Fig. 7. Analytic residual bound after one reciprocal table reduction. More table bits shrink the interval used by the polynomial.

13.3 Approximate the residual function

After the table reduction, the remaining function is:

$$\log_2(1+u)$$

Since u is small, approximate:

$$g(u) = \frac{\log_2(1+u)}{u}$$

Then:

$$\log_2(1+u) \approx u\,p(u)$$

Dividing by u removes the zero at the origin. The resulting function $g(u)$ is smooth near zero and has value $1/\log(2)$.

A Sollya script for the polynomial looks like this:

```
prec = 200!;
display = hexadecimal!;

U = [-1/32; 1/32];
g = log2(1 + x) / x;

p = fpminimax(g, 5, [|D...|], U, relative);

abs_err = supnorm(x * p, log2(1 + x), U, absolute, 1b-80);
rel_err = supnorm(p, g, U, relative, 1b-80);

print("absolute residual error:", abs_err);
print("relative kernel error:", rel_err);
print("polynomial:");
horner(p);
```

The script stops at the polynomial boundary. It does not validate table indexing, split constants, or the final addition with the exponent.

13.4 Generate the table

Generate tables from the invariant, not from hand-edited rows.

For each bucket, generate:

$$c_i = 1 + \frac{i + 1/2}{B}$$
$$r_i = \mathrm{round}_D\left(\frac{1}{c_i}\right)$$
$$a_i = -\log_2(r_i)$$

Then split a_i into a high part and a low part:

$$h_i = \mathrm{round}_t(a_i)$$
$$\ell_i = \mathrm{round}_D(a_i - h_i)$$

Here t is the chosen precision for the high part. It should be chosen so that the later addition with the exponent is exact for the path being proved.

A Sollya table script can be shaped like this:

```
prec = 200!;
display = hexadecimal!;

B = 16!;
hi_bits = 32!;

for i from 0 to B - 1 do {
  c = 1 + (i + 0.5) / B;
  r = round(1 / c, D, RN);
  a = -log2(r);
  hi = round(a, hi_bits, RN);
  lo = round(a - hi, D, RN);
  print("{", r, ",␣", hi, ",␣", lo, "},");
};
```

Record B, `hi_bits`, the residual interval, and the Sollya version in the generation log.

Running it prints the table rows that the kernel below indexes into:

```
{ 0x1.f07c1f07c1f08p-1 , 0x1.6bad3758p-5 , 0x1.dfb02d7c85b4bp-38 },
{ 0x1.d41d41d41d41dp-1 , 0x1.08c588cep-3 , -0x1.6186b2964f9cdp-37 },
{ 0x1.bacf914c1badp-1 ,  0x1.acf5e2dcp-3 , -0x1.626dde58f9bb3p-36 },
{ 0x1.a41a41a41a41ap-1 , 0x1.24407abp-2 ,  0x1.c0e74d235067fp-35 },
{ 0x1.8f9c18f9c18fap-1 , 0x1.6e221cdap-2 , -0x1.799147a3bbf43p-37 },
{ 0x1.7d05f417d05f4p-1 , 0x1.b47ebf74p-2 , -0x1.df57bfe59cdc9p-36 },
{ 0x1.6c16c16c16c17p-1 , 0x1.f7a8568cp-2 ,  0x1.60d9ba05e404ep-35 },
{ 0x1.5c9882b931057p-1 , 0x1.1bf311eap-1 , -0x1.45fe39700e4e8p-34 },
{ 0x1.4e5e0a72f0539p-1 , 0x1.3abb3faap-1 ,  0x1.0b3eb1fb3a62fp-40 },
```

```
{ 0x1.4141414141414p-1 , 0x1.5848226ap-1 , -0x1.d8b30391f958cp-35 },
{ 0x1.3521cfb2b78c1p-1 , 0x1.74b1fd64p-1 ,  0x1.c0ea88bb501d9p-34 },
{ 0x1.29e4129e4129ep-1 , 0x1.900e616p-1 ,   0x1.66cfec74663c3p-44 },
{ 0x1.1f7047dc11f7p-1 ,  0x1.aa708f58p-1 ,  0x1.4d4358872f1c7p-41 },
{ 0x1.15b1e5f75270dp-1 , 0x1.c3e9ca2ep-1 ,  0x1.a0553ef42f00ep-37 },
{ 0x1.0c9714fbcda3bp-1 , 0x1.dc899ab4p-1 , -0x1.5289d70b977e4p-42 },
{ 0x1.041041041041p-1 ,  0x1.f45e08bcp-1 ,  0x1.e0cac0def2e1dp-34 },
```

Both design choices can be read straight off the rows. Every high part ends in at least 20 zero bits because it was rounded to 32 bits, which is what makes the later addition exact. Every low part is around 2^{-34} or smaller, which is the resolution the high part left behind. Checking generated constants against the invariants this way catches script bugs before they become wrong answers.

13.5 Kernel shape

Keep the code close to the derivation. The snippet below omits the actual generated table rows, but it preserves the data flow that has to be proved:

```
struct Log2TableEntry
{
    double r;
    double hi;
    double lo;
};

struct Log2Poly
{
    double c1;
    double c2;
    double c3;
    double c4;
    double c5;
};

[[nodiscard]] constexpr double log2_core_small(double m,
                                               int e,
                                               const Log2TableEntry* table,
                                               const Log2Poly coeff) noexcept
{
    // m must lie in [1, 2).  The clamp guards the top edge so a caller
    // bug cannot index past the table.
    int i = static_cast<int>((m - 1.0) * 16.0);
    if (i > 15) { i = 15; }
    const Log2TableEntry entry = table[i];
```

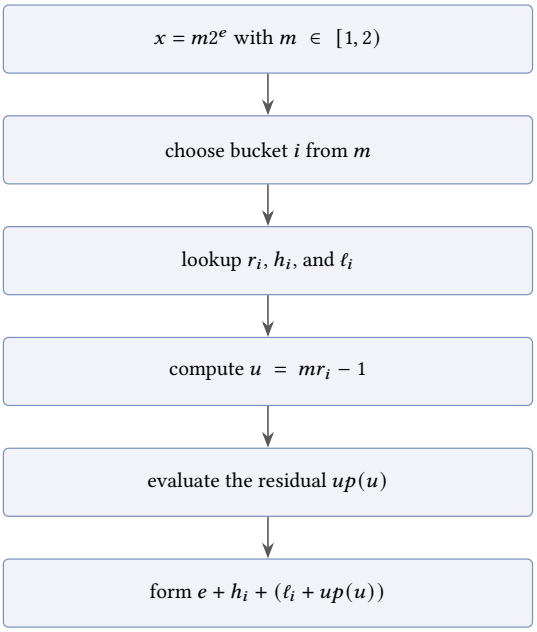

Fig. 8. Data flow in the small $\log_2$ core.

```
    const double u = std::fma(m, entry.r, -1.0);
```

The index computation needs more care than it looks like it should. The cast truncates toward zero, so m exactly at a bucket boundary lands in the upper bucket, and the residual bound must hold for that assignment. Production kernels usually derive the index from the top mantissa bits with integer operations instead of floating-point arithmetic, which makes the boundary behavior exact by construction and removes the clamp.

```
    const double p =
        (((coeff.c5 * u + coeff.c4) * u + coeff.c3) * u + coeff.c2) * u
        + coeff.c1;

    const double residual = u * p;
    const double tail = entry.lo + residual;

    return (static_cast<double>(e) + entry.hi) + tail;
}
```

The coefficient object is produced from the Sollya polynomial. The table object is produced by the table-generation script. Production code should store those generated values directly in the implementation or in the nearby generated constants file.

13.6 What must be proved

The table core has more obligations than the small sine kernel. The proof needs bounds for:

(1) table index selection
(2) reciprocal-table rounding
(3) the size of u
(4) the polynomial approximation to $\log_2(1+u)$
(5) the floating-point evaluation of $u\,p(u)$
(6) the split addition $e + h_i + \ell_i$
(7) the final rounding under each supported mode

A Gappa proof is a good fit for the straight-line part after the table entry has been chosen. MPFR testing should cover bucket boundaries, values where u is near its largest magnitude, exact powers of two, subnormals routed through the full log path, and every supported rounding mode.

13.7 What this example leaves out

A complete function still needs:

(1) bit decomposition for all finite positive inputs
(2) handling of zero, negative inputs, NaN, and infinities
(3) subnormal normalization
(4) a larger or more carefully designed table
(5) directed rounding checks
(6) an accurate path for hard cases if correct rounding is the target

Real kernels use this same shape: reduce with a table, keep the residual small, approximate a smooth transformed function, split constants for exactness, and prove the operation sequence that is actually compiled.

Checkpoint

Three questions about the table reduction.

(1) What does the reciprocal table actually buy?
(2) Why is $-\log_2(r_i)$ stored as a high part and a low part instead of one double?
(3) Why does the polynomial approximate $\log_2(1+u)/u$ rather than $\log_2(m)$ on $[1, 2)$ directly?

Answers. The table shrinks the interval the polynomial must cover, which lowers the degree and tightens the error bound. The high part is rounded to fewer bits so that adding it to the exponent term is exact, while the low part carries the remaining bits through the compensated tail. The residual u lives in a small interval around zero, where a low-degree polynomial is accurate, and dividing by u removes the zero so relative error stays stable.

Milestone. You can now open a table-driven kernel in any libm source tree and name what each constant is for: which value it approximates, how it was split, and which later operation its rounding was chosen to keep exact.

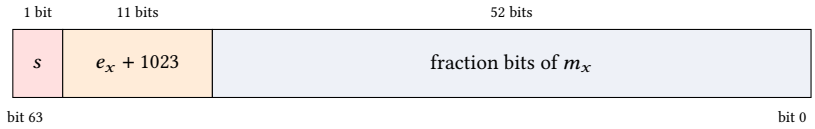

Fig. 9. The 64-bit double layout, field widths not to scale. The exponent is stored with a bias so it is always nonnegative, and the leading 1 of the significand is implicit. The algebraic split $s_x \cdot 2^{e_x} \cdot (1 + m_x)$ reads straight out of these three fields.

14 The sign, exponent, and significand split

For normal finite binary floating-point values, use the decomposition [14, 18]:

$$x = s_x \cdot 2^{e_x} \cdot (1 + m_x)$$

This decomposition applies to finite nonzero normal binary floating-point numbers.

- s_x is the sign, either $+1$ or -1
- e_x is the unbiased binary exponent
- m_x is the fractional part of the significand
- $1 + m_x$ is the normalized significand and lies in $[1, 2)$

The stored bits mirror the algebra directly. Figure 9 shows the 64-bit double layout. The exponent field holds e_x plus a bias of 1023 so it can be stored unsigned. The leading 1 of the normalized significand is not stored at all, since normal numbers always have it. This layout is why the split is cheap: extracting e_x is a shift and a mask on the bit pattern, with no arithmetic error anywhere.

Example:

$$6.5 = 1.625 \cdot 2^2$$

so:

$$s_x = +1$$
$$e_x = 2$$
$$m_x = 0.625$$
$$6.5 = +1 \cdot 2^2 \cdot (1 + 0.625)$$

In binary:

$$6.5 = 110.1_2 = 1.101_2 \cdot 2^2$$

The exponent carries scale. The significand stays in a fixed interval. That turns a many-magnitude problem into a smaller approximation problem plus integer exponent bookkeeping.

The $(1 + m_x)$ form is for normal numbers. Subnormal numbers do not have the implicit leading 1 and need normalization or special handling. Zero, infinity, and NaN do not fit this decomposition [14]. Negative bases for pow need integer-exponent handling before any logarithmic path.

15 Argument reduction

Approximation works well only on a small interval. Argument reduction is how the implementation gets there [17, 18].

For `exp`:

$$\exp(x) = 2^k \exp(r)$$

where:

$$k \approx \text{round}\left(\frac{x}{\log(2)}\right)$$

and:

$$r = x - k\log(2)$$

The integer k carries the power-of-two scale. The residual r is small enough for a short polynomial.

For `exp2`, the split is simpler:

$$2^x = 2^k 2^r$$

where:

$$k = \lfloor x \rfloor$$

and:

$$r = x - k$$

The reconstruction step puts 2^k back into the exponent field when the result is in range.

For $\log_2$, the sign, exponent, and significand split gives:

$$\log_2(x) = e_x + \log_2(1 + m_x)$$

For natural log:

$$\log(x) = e_x \log(2) + \log(1 + m_x)$$

The exponent term is exact integer bookkeeping. The hard part is approximating the significand term, where $(1 + m_x)$ lies in a fixed interval for normal inputs.

Log implementations therefore usually extract the exponent first, reduce the significand, and approximate only a small residual.

For trig functions, the usual reduction is modulo $\pi/2$ or $\pi/4$, followed by quadrant reconstruction.

Trig reduction is hard for large inputs. π is irrational. If x is huge and close to a multiple of $\pi/2$, subtracting $k\pi/2$ can lose the useful bits.

15.1 Cody-Waite reduction

Cody-Waite reduction is the moderate-input reducer. It represents the reduction constant as a small expansion of machine numbers:

$$C = C_{\text{hi}} + C_{\text{mid}} + C_{\text{lo}}$$

$$r = x - kC_{\text{hi}} - kC_{\text{mid}} - kC_{\text{lo}}$$

The split is there to control where rounding first enters the calculation. A single rounded constant can already be too crude. For example, reduce $x = 30$ for `exp`. The nearest double to $\log(2)$ is about $9.4 \cdot 10^{-18}$ below the real value. With $k = 43$, the residual inherits roughly $4.0 \cdot 10^{-16}$ of constant error before the floating-point operations have rounded anything. Around $r \approx 0.1947$, that is already about 15 ULPs. The polynomial cannot repair bits the reducer never delivered.

The useful Cody-Waite invariant is exactness. Choose C_{hi} with enough trailing zero bits that kC_{hi} is exactly representable for every k produced by the fast reducer. Then arrange the reduction interval so x and kC_{hi} are close. Sterbenz's lemma makes the first subtraction exact, so the leading shared bits cancel without injecting a new rounding error. The smaller terms kC_{mid} and kC_{lo} then put back the pieces of C that C_{hi} deliberately omitted. Figure 10 shows the bit alignment rather than the arithmetic syntax.

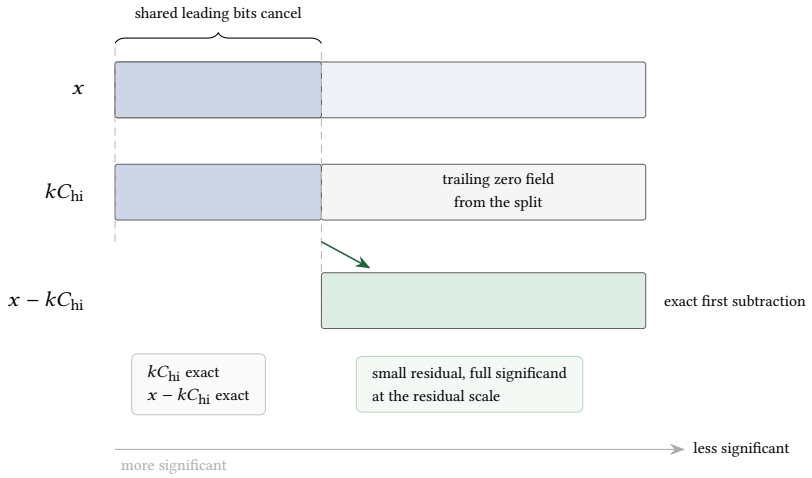

Fig. 10. Cody-Waite as a bit-alignment argument. The high split is chosen so kC_{hi} is exact. When it shares the leading bits of x, the first subtraction is exact and the residual begins at the lower-significance position where the cancellation ended.

Write the exactness conditions into the design note, because each one can fail without an obvious local symptom. The integer k must stay inside the range used when choosing the trailing zeros of C_{hi}. The values x and kC_{hi} must be within a factor of two for Sterbenz's lemma to apply [18]. The residual must also stay large enough, relative to the discarded kC_{lo} tail, that the later additions do not erase the bits the split preserved. All of these are checkable from the reducer's input range.

15.2 Payne-Hanek reduction

Payne-Hanek reduction is the large-input reducer for trig functions. It uses a long table of the reduction constant and a fixed-precision integer multiply [17].

Ordinary double arithmetic is not strong enough to reduce every binary64 value modulo $\pi/2$. Some inputs land extraordinarily close to a nonzero multiple of $\pi/2$. A direct subtraction then has to cancel sixty or more leading bits and still leave a trustworthy residual. That is outside the useful range of a short Cody-Waite split.

The Payne-Hanek route computes the fractional part of $x \cdot \frac{2}{\pi}$ instead of subtracting $k\pi/2$ directly [21]. The table stores $2/\pi$ as integer words, with comfortably more than a thousand bits for a binary64 implementation. Since x is a significand times a power of two, its exponent selects the only window of table words that can influence the quadrant and the high bits of the residual. Words above that window contribute only discarded multiples of 4. Words below it cannot reach the rounding decision.

The core operation is an integer product: the 53-bit significand of x times the selected table window. The two bits around the binary point give $k \bmod 4$. The remaining fractional part is converted back to floating point and multiplied

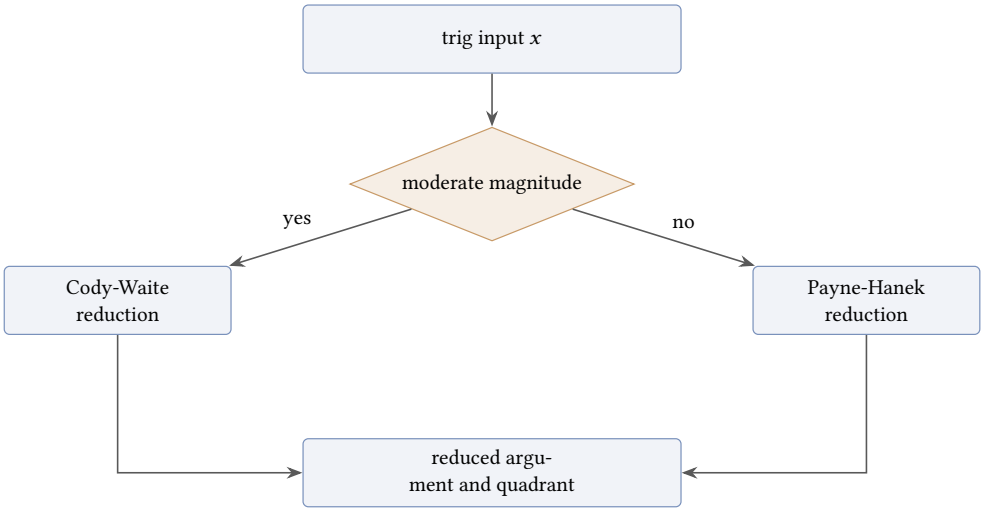

Fig. 11. Typical trig reduction split between moderate and large inputs.

$k \bmod 4$	$\sin(x)$ reconstruction
0	$\sin(r)$
1	$\cos(r)$
2	$-\sin(r)$
3	$-\cos(r)$

by $\pi/2$ to form the reduced argument. Guard bits in the window are sized from the worst possible cancellation. For binary64, that worst case is known by exhaustive search over the exponent range, which fixes the required window once rather than by guesswork [17].

15.3 Trig quadrant reconstruction

$$x = k\frac{\pi}{2} + r$$

The reducer returns an integer k and a small residual r. The value of $k \bmod 4$ decides which small kernel reconstructs the result [17].

Figure 12 shows where the table comes from. Reducing modulo $\pi/2$ lands the angle in one of four quadrants of the unit circle. In each quadrant, the sine of the full angle equals plus or minus the sine or cosine of the small residual, so two kernels and a sign cover the whole circle.

A full trig function is therefore more than one polynomial. The reducer and quadrant logic choose the kernel and the returned sign.

15.4 Reduction error

Reduction has its own error.

If the polynomial was proved for an exact reduced argument r, but the implementation computes r with too much error, the proof no longer applies.

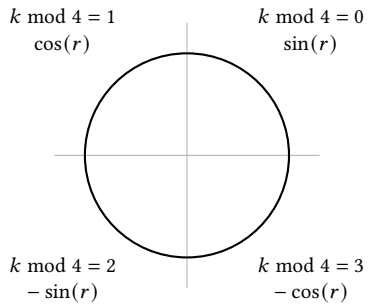

Fig. 12. Quadrant reconstruction for $\sin(x)$. The reducer writes $x = k\pi/2 + r$, the quadrant index $k \bmod 4$ picks the kernel and the sign, and the polynomial only ever evaluates near zero.

Decide the total error target first. Give part of that budget to reduction. The reducer should carry enough extra precision that its error stays below that budget.

For many functions, this means split constants or a double-word reduced argument. For very large trig inputs, it can mean a separate large-argument reducer.

Checkpoint

This one is a pencil-and-paper exercise. Reduce $x = 1000$ for exp using one rounded constant L for $\log(2)$. Compute k, then the drift $k(\log 2 - L)$, then express the drift in ULPs of the residual. The numbers from Section 8 and the worked $x = 30$ case above contain everything needed.

Answer. $k = 1443$, the residual is $r \approx -0.2114$, and the drift is $1443 \cdot 9.4 \cdot 10^{-18} \approx 1.36 \cdot 10^{-14}$, which is about 490 ULPs of r. The input grew 33 times over the $x = 30$ case and the drift grew with it, because the drift scales with k. That scaling is why the split is not optional.

16 Evaluating the polynomial

After Sollya gives coefficients, the implementation still has to evaluate the polynomial in floating point.

The mathematical polynomial and the C++ expression are different objects. Every floating-point operation can round.

16.1 Horner evaluation

Horner evaluation is the usual default:

$$p(x) = c_0 + x(c_1 + x(c_2 + x(\cdots)))$$

It uses few operations and maps well to FMA [17, 18].

With FMA, one step can be:

```
r = fma(r, x, c);
```

That rounds the multiply-add once.

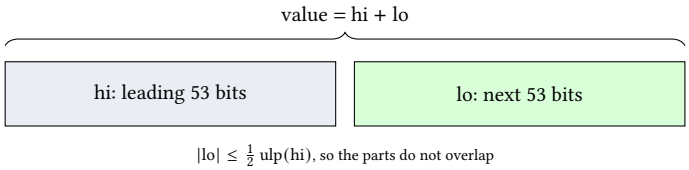

Fig. 13. A double-word value. Two ordinary doubles, kept as an unevaluated sum, act as one number with about twice the significand bits. No new hardware type is involved, only bookkeeping.

`std::fma` comes with caveats. Unless the build pins `FP_CONTRACT` or the equivalent flag, a compiler may fuse a written `a*b + c` on its own, or decline to, so write `std::fma` where the proof needs one rounding and separate operations where it needs two. On targets without an FMA instruction the call is emulated in software, still correctly rounded but far slower, so check the target before designing a kernel around dozens of them. And `std::fma` is usable in constant expressions only from C++23, while compile-time arithmetic always rounds to nearest, so a constexpr kernel can legitimately produce different bits than the same kernel running under a directed mode.

16.2 Estrin evaluation

Estrin evaluation groups terms so more work can happen in parallel [18].

Example:

$$c_0 + c_1x + c_2x^2 + c_3x^3$$

can be grouped as:

$$(c_0 + c_1x) + x^2(c_2 + c_3x)$$

This can be faster on wide cores. It can also need more temporaries. Use it when measurements show that it helps.

16.3 Double-word evaluation

Some kernels need more precision in part of the evaluation.

Common tools include Fast2Sum, 2Sum, 2MultFMA, Veltkamp-Dekker splitting, double-double arithmetic, and compensated Horner evaluation [17, 18].

The common trick is to represent one value as an unevaluated sum of two doubles. The high part carries the leading 53 bits, the low part carries the next 53, and the pair behaves like a number with roughly 106 significand bits as long as every operation tracks both pieces. Figure 13 shows the layout. The error terms that Fast2Sum and 2MultFMA recover are exactly the low parts this representation needs.

Do not use double-double everywhere by default. Use it where the error budget needs it.

The most useful extra precision is usually local. It protects one reduction, one product, or one reconstruction step. Treat it as a tool for a specific bound, not as a blanket fix.

17 Fast precision and accurate precision

Many elementary functions use more than one precision tier.

Keep the fast path cheap. It handles ordinary inputs where a modest amount of extra precision is enough. The accurate path handles cases where the result is too close to a rounding boundary, or where the fast path cannot prove the final rounding decision.

Tier	Role
double or double-double-like	Fast path
128-bit dyadic	First accurate path
256-bit dyadic	Hard-case path when needed

For float functions, the implementation can often use double or double-double arithmetic for the fast path. Some cases may need a wider accurate path.

For double functions, the fast path usually needs a double-double-like computation. That does not mean every operation should behave like a complete higher-precision type. A full double-double type that renormalizes after every operation is often too expensive for the hot path.

A faster pattern is to arrange the computation so that important intermediate pieces are exact, or have a known error, and normalize only where the bound needs it. This is less general than a full double-double type, but it fits a fixed numerical kernel better.

The accurate path may use a dyadic floating-point type with a fixed integer significand, such as a 128-bit or 256-bit dyadic format. This gives a large exponent range and controlled precision without relying on hardware quad precision.

Quad precision and double-double are not interchangeable.

Quad precision has a much larger exponent range than double-double. A pair of doubles gives extra significand bits, but it does not cheaply give the exponent range of a true quad format. A dyadic format can be a better accurate-path type when the algorithm needs wide range and predictable fixed precision.

One tiering pattern is:

This is especially relevant for pow. A double pow implementation is harder than a float pow implementation because the final result has a 53-bit significand. The intermediate log, multiply, and exp reconstruction must leave enough margin for correct rounding.

18 Lookup tables

Lookup tables in elementary functions are not only cached constants.

A table entry usually exists to make one reduction step cheaper, more accurate, or both. For log and pow, common table values include approximations to quantities like $-\log_2(r)$, split into pieces.

A split table might store:

$$a = -\log_2(r)$$

and:

$$a = a_{\text{hi}} + a_{\text{mid}} + a_{\text{lo}}$$

Choose the split for a reason. A high part may be rounded to a smaller precision so that a later multiplication is exact, or so that adding it to an exponent does not lose important bits.

Table generation should start from an invariant, not from a literal list of constants.

A table-generation note should record:

(1) how the index is chosen
(2) what real value each entry approximates
(3) how the entry is split
(4) which later operation is meant to be exact or tightly bounded

(5) what interval remains after the table reduction
(6) which precision tier uses the table

For powf, the fast path may use tables split into double-double or triple-double pieces because the target result is float. For pow in double, the accurate paths may need tables generated directly for the wider dyadic type used in that path.

Do not assume one table layout fits every precision tier.

If the accurate pass uses a 128-bit dyadic format, generate a table suited to that format. If there is a 256-bit hard-case path, generate the table for that path as well. The values may represent the same mathematical constants, but the split, precision, and exactness constraints can be different.

Without its generation rule, a table is hard to audit. With the invariant recorded, it is part of the proof.

18.1 Log table reduction

$$\log_2(1 + m_x) = -\log_2(r_i) + \log_2(r_i(1 + m_x))$$

Define:

$$u = r_i(1 + m_x) - 1$$

Then:

$$\log_2(1 + m_x) = -\log_2(r_i) + \log_2(1 + u)$$

The table value r_i is chosen so that u is small. The polynomial only approximates $\log_2(1 + u)$ on a smaller interval. The table entry supplies the correction term $-\log_2(r_i)$.

That is the value of the table. It does more than avoid a computation. It makes the residual small enough for a cheaper and more accurate polynomial. Section 13 uses the same pattern with bucket reciprocals and split $-\log_2(r_i)$ constants.

19 Why pow is harder than exp or log

pow combines the difficulties of log and exp.

For positive finite x, the usual shape is:

$$\mathrm{pow}(x, y) = \exp(y \log(x))$$
$$\mathrm{pow}(x, y) = 2^{y \log_2(x)}$$

$$y \log_2(x) = y(e_x + \log_2(1 + m_x))$$

That is why pow implementations often contain a log core, table reduction for the significand, split constants, and extra precision around the product $y \log_2(x)$. A small error in the log value can be magnified by y, then carried into the exp reconstruction. Figure 14 shows the chain.

That hides several problems.

First, $\log(x)$ needs its own range reduction, tables, approximation, and reconstruction.

Second, $y \log(x)$ can magnify error. The relative error of the final result is roughly $\log(2)$ times the absolute error of the product $y \log_2(x)$, so landing within 2^{-60} of the true result means knowing that product to about 2^{-60} absolutely. When y is as large as 2^{10}, the log core must deliver $\log_2(x)$ to about 2^{-70}, seventeen bits past what a plain double can carry. This is why pow cores keep the log in double-double or wider from the start.

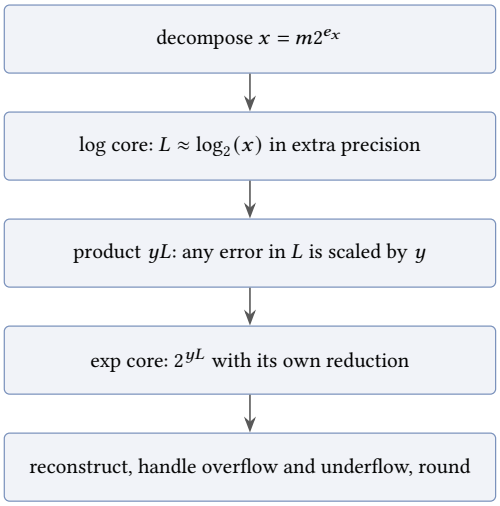

Fig. 14. pow as a composition. Each core is a full reduction, approximation, and reconstruction pipeline of its own, and the product in the middle amplifies whatever error the log core left behind.

Input	Required result
$\mathrm{pow}(x, \pm 0)$	1 for every x, including NaN
$\mathrm{pow}(1, y)$	1 for every y, including NaN
$\mathrm{pow}(-1, \pm\infty)$	1
$\mathrm{pow}(\pm 0, y)$, y a negative odd integer	$\pm\infty$, divide-by-zero
$\mathrm{pow}(\pm 0, y)$, y a positive odd integer	± 0
$\mathrm{pow}(x, y)$, finite $x < 0$, non-integer y	NaN, invalid
$\mathrm{pow}(x, -\infty)$, $\lvert x\rvert < 1$	$+\infty$
$\mathrm{pow}(x, +\infty)$, $\lvert x\rvert < 1$	$+0$

Third, exp has to reconstruct the final result, including overflow and underflow behavior.

Fourth, pow has many special cases. The C++ contract pins a long list of them exactly, and several are easy to guess wrong:

The table is abridged. The full list lives in Annex F of the C standard and transfers to C++ through <cmath>. Exact and near-exact cases also need dedicated paths: integer exponents of moderate size, bases that are powers of two, and $y = 0.5$, which agrees with sqrt everywhere except that $\mathrm{pow}(-0, 0.5)$ returns $+0$ while $\mathrm{sqrt}(-0)$ returns -0.

Correct rounding compounds all of it. Double-precision pow stayed out of reach for production libraries far longer than exp or log, and the CORE-MATH project documents both the algorithms and the remaining worst-case questions involved [28].

Because of this, pow is not a good first elementary function.

A better path is:

(1) implement one exp-family function
(2) implement one log-family function
(3) learn the reduction, table, polynomial, and rounding structure

Hazard	Why it matters
FMA contraction	`a*b + c` may become one fused operation
Reassociation	fast-math may reorder arithmetic
Default rounding assumptions	directed modes may be ignored
Hidden precision	intermediates may have different width
Floating-point environment access	`fenv` support differs by compiler

(4) then implement pow

`pow` is best understood as a composition of those two families, plus a large special-case layer and a harder final-rounding problem.

20 Compiler behavior

An error proof assumes a specific sequence of floating-point operations. The compiler may change that sequence unless the source and build settings prevent it.

The main hazards are worth naming explicitly:

These two expressions do not have the same rounding behavior:

```
a * b + c
```

```
std::fma(a, b, c)
```

The first can round the product and then the sum. The second rounds once.

Pitfall: proving one program and compiling another

If a proof needs FMA, use FMA explicitly. If a proof needs the live rounding mode, compile in a mode that respects it. Do not prove one operation sequence and compile a different one [18]. Fast-math flags break the bounds in this guide.

This also affects double-double arithmetic. Some exact multiplication algorithms for double-double arithmetic without FMA are only valid under round-to-nearest. Making them correct under directed rounding can require extra handling. That cost belongs in the design.

20.1 The flags that matter

The hazards above map onto a small set of compiler switches. GCC and Clang spellings are shown. MSVC bundles roughly the same choices into `/fp:fast`, `/fp:precise`, and `/fp:strict`.

For library work, keep the policy simple: build with precise semantics, state the flags next to every measured number, and treat a user who links the kernel into a fast-math build as outside the proof.

21 The error budget

$$E_{\text{total}} \leq E_{\text{reduce}} + E_{\text{approx}} + E_{\text{eval}} + E_{\text{reconstruct}} + E_{\text{round}}$$

Flag	Effect on a math kernel
`-ffast-math`	Bundle of promises: reassociation, no signed zeros, no NaN or infinity handling, reciprocal shortcuts, and on some targets flushing subnormals to zero. Any one of them invalidates an IEEE-based proof. Defines `__FAST_MATH__`, which a header can detect and warn on.
`-ffp-contract=fast`	Lets the compiler fuse `a*b + c` even outside fast math. It is the default on some compiler and target combinations, so pin it rather than assume it.
`-fno-math-errno`	Stops libm calls from maintaining `errno`. The C contract allows error reporting through `errno`, through floating-point flags, or both, advertised by `math_errhandling`. Maintaining `errno` blocks vectorization and inlining, which is why many libraries document that they never set it.
`-frounding-math`	Tells the compiler not to assume round to nearest, which disables constant folding and code motion that would break `fesetround` callers.
`-fexcess-precision=standard`	On targets that evaluate in wider registers, x87 being the classic case, forces predictable rounding of intermediates. `FLT_EVAL_METHOD` reports the policy. Double rounding through a wider format can change results, so platforms where it is nonzero need their own validation pass.

Term	Bound	Share of 2^{-54}
E_{reduce}	2^{-60}	about 2%
E_{approx}	2^{-58}	about 6%
E_{eval}	2^{-57}	about 12%
$E_{\text{reconstruct}}$	2^{-59}	about 3%
total	$15 \cdot 2^{-60}$	under a quarter

- E_{reduce} comes from argument reduction
- E_{approx} comes from replacing the function with a polynomial or rational function
- E_{eval} comes from evaluating that approximation in floating point
- $E_{\text{reconstruct}}$ comes from scaling, sign changes, table corrections, and final algebra
- E_{round} is the margin needed for the final rounding decision

Use this inequality as a model for thinking about the implementation. It is not always the final tight proof. It prevents a common mistake: proving the polynomial and forgetting the rest of the function.

A small worked allocation makes the model concrete. Suppose a double function targets faithful rounding, so everything before the final rounding must stay below half an ULP. Within the binade $[1, 2)$ that is at worst 2^{-54} in relative terms. One workable split:

These numbers are illustrative rather than prescriptive. What matters is the shape of the allocation. Each stage gets an explicit slice, the slices sum with margin to spare, and a stage that cannot meet its slice forces a redesign before any code is written.

Mode	C++ `fenv` constant	Guarantee
Round to nearest, ties to even	`FE_TONEAREST`	closest neighbor
Round toward positive infinity	`FE_UPWARD`	never below the exact result
Round toward negative infinity	`FE_DOWNWARD`	never above the exact result
Round toward zero	`FE_TOWARDZERO`	magnitude never grows

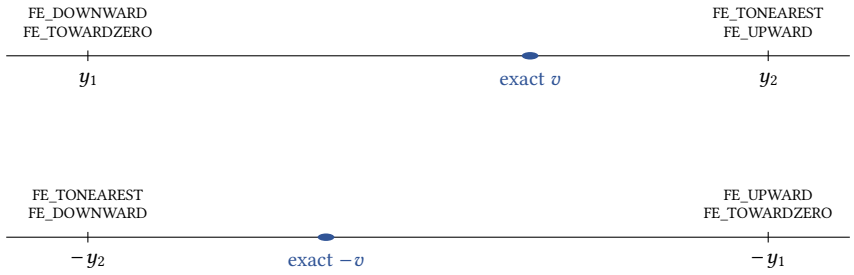

Fig. 15. What each mode returns when the exact result v falls between the representable neighbors $y_1 < y_2$, shown for v and for $-v$. Round to nearest ignores the sign. The directed modes do not: round toward zero picks the lower neighbor for positive values and the upper neighbor for negative ones.

22 Rounding modes

Every floating-point operation ends with the same step. The arithmetic produces a value that is usually not representable, and a rule picks which representable neighbor to return. IEEE 754 defines four such rules for binary formats, and C++ exposes them through `<cfenv>` [14].

Round to nearest is the default on every mainstream platform and the only mode most code ever runs in. When the exact result lands exactly halfway between two neighbors, the tie goes to the neighbor whose last significand bit is zero. The tie rule earns its keep in long computations: always rounding ties upward would push results steadily in one direction, while ties to even alternates the direction and the bias cancels on average [18].

The three directed modes trade closeness for a guaranteed direction. That guarantee is what interval arithmetic buys: run a computation once rounding down and once rounding up, and the true result is bracketed between the two outputs. Directed modes also appear in validated numerics and in tests that hunt for rounding sensitivity. Figure 15 shows all four decisions on one input and its negation.

22.1 The C++ mechanics

Rounding mode is dynamic state, not a property of the source code. `std::fesetround` changes it for the calling thread, `std::fegetround` reads it, and any function may be called under whatever mode the caller installed. Three consequences matter for library work.

First, a function does not get to choose the mode it runs under. If the caller installed `FE_UPWARD`, every add, multiply, and FMA inside the function rounds upward, whether the author planned for that or not.

Second, compile-time arithmetic ignores the dynamic mode. Constant folding and `constexpr` evaluation round to nearest. The same expression can therefore produce different bits at compile time and at run time, and a cached compile-time constant can disagree with the runtime computation that recreates it.

Third, the optimizer assumes the default mode unless told otherwise. The `#pragma STDC FENV_ACCESS` switch exists for this, but support is uneven across compilers, so code that mixes `fesetround` with ordinary arithmetic needs testing on every toolchain it claims to support.

The effect is easy to reproduce. The program `tools/proofs/math/approx_doc/cpp/directed_rounding_demo.cpp` evaluates two expressions under each mode, compiled with `-frounding-math` so the compiler does not fold them:

```
FE_TONEAREST   1/3 = 0x1.5555555555555p-2   0.1+0.1+0.1 = 0x1.3333333333334p-2
FE_UPWARD      1/3 = 0x1.5555555555556p-2   0.1+0.1+0.1 = 0x1.3333333333334p-2
FE_DOWNWARD    1/3 = 0x1.5555555555555p-2   0.1+0.1+0.1 = 0x1.3333333333333p-2
FE_TOWARDZERO  1/3 = 0x1.5555555555555p-2   0.1+0.1+0.1 = 0x1.3333333333333p-2
```

The quotient moves by one bit under `FE_UPWARD` and the repeated sum moves under the two downward modes. The source is identical in every row, only the installed mode differs, and a library function has to be correct under all four.

22.2 Why round-to-nearest proofs do not transfer

A bound proved under round to nearest does not automatically hold in the directed modes.

The most direct problem is the unit rounding error, which doubles. One operation is off by at most half an ULP under round to nearest, but by up to a full ULP under a directed mode, so every term of an error budget grows.

A subtler problem is that building blocks such as Fast2Sum, 2Sum, and Veltkamp splitting are theorems about round to nearest [17, 18]. Under directed rounding, some of them silently stop producing the exact sums and products their names promise, and any proof built on them collapses.

Long chains of operations also behave differently. Under round to nearest, individual rounding errors point in both directions and partially cancel. Under a directed mode every error points the same way, so the computation drifts steadily to one side of the true value instead of wobbling around it.

Signed zeros are a final trap. A result that is exactly zero must carry a sign, and which sign the standard requires can depend on the active mode. Write the special-case table per mode, not once.

23 Correct rounding

Correct rounding is the strongest result: every input returns the value required by the active rounding mode.

The exact result can land extremely close to a rounding boundary. In that case, the implementation needs enough extra precision to know which side of the boundary the exact result is on. This is called the Table Maker's Dilemma [17, 18].

Figure 16 shows the situation. The implementation never has the exact result. It has a computed value and a proven bound, which together pin the exact result inside an interval. If that interval sits entirely on one side of the rounding boundary, the rounding decision is forced and the fast answer is correct. If the interval straddles the boundary, the fast result cannot settle the question, and the implementation has to recompute with a tighter bound.

23.1 Faithful rounding

Faithful rounding relaxes that requirement. The returned value may be either of the two floating-point neighbors around the exact result.

Faithful rounding can be a useful interim target. It is not the same as correct rounding.

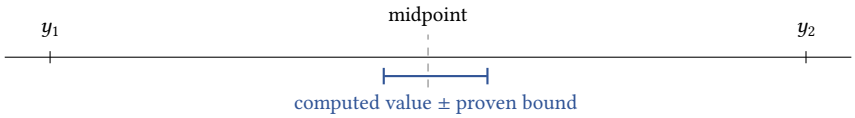

Fig. 16. A hard case under round to nearest. y_1 and y_2 are consecutive representable values. The proven error bound pins the exact result inside the marked interval, but the interval straddles the midpoint, so the rounding decision is unknown and a more precise path must run.

23.2 Ziv's strategy

Ziv's strategy uses a fast path and a fallback [17, 18, 29]. Most inputs take the fast path. Rare hard cases take the accurate fallback. Correctly rounded libraries such as CRlibm use this pattern [6].

23.3 Hard cases

Hard cases turn up even in small samples. The script `tools/proofs/math/approx_doc/sollya/hard_case_scan.sollya` walks a grid of 4097 doubles, evaluates `exp` of each to 300 bits, and measures the distance from each exact result to its nearest rounding boundary:

```
samples: 4097
hardest input found: 0.4664154052734375
distance to rounding boundary in ulps: 8.2133230542e-5
extra bits needed to decide: 13.57
```

Four thousand ordinary inputs already contain one whose rounding decision needs the exact result to about 14 bits past the 53-bit format. Scale the sample up and the worst case keeps getting worse, roughly one extra bit per doubling. Exhaustive searches over the entire binary64 format, pioneered by Lefèvre and Muller, find inputs for `exp` and its relatives that need the result to more than 100 bits in total before the decision is safe [16].

That search result is what sizes the accurate path. A second phase that computes to 120 bits, with a proven bound, decides every known hard case for the functions whose searches are complete, which is how correctly rounded libraries pick their fallback precision [6, 17].

For some functions and formats the search is still open, especially for binary128 and for two-argument functions like `pow`, where algorithms such as SLZ push the frontier [17, 18]. When the worst case is unknown, a library can still be correct by iterating Ziv's strategy with growing precision, but it cannot promise a bound on the slowest call.

24 Special cases

Special cases are part of the function.

Handle NaNs, infinities, signed zeros, subnormals, exact results, overflow, underflow, invalid operations, and divide-by-zero behavior before the approximation path.

Keep the polynomial kernel inside the range where its algebra is valid.

For each function, write the special-case table before writing the polynomial. Here is what that table looks like for binary64 `exp` under round to nearest:

Input	Result	Flags
quiet NaN	the same NaN	none
signaling NaN	a quiet NaN	invalid
$+\infty$	$+\infty$	none
$-\infty$	$+0$	none
± 0	1, exactly	none
x above about 709.8	$+\infty$	overflow, inexact
x between about -745.2 and -708.4	a subnormal	underflow, inexact
x below about -745.2	$+0$	underflow, inexact
$\lvert x\rvert$ below about 2^{-54}, nonzero	1	inexact

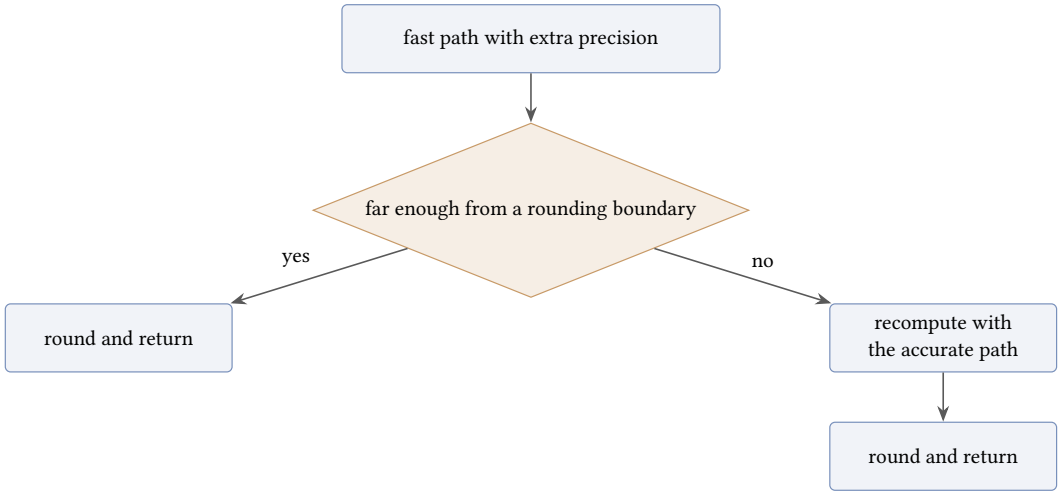

Fig. 17. Ziv's strategy as a fast-path test with a precise fallback.

Each threshold is a specific double that the implementation must get right, and the approximate values above are only signposts. The table also shifts with the rounding mode. Under `FE_DOWNWARD` an overflowing `exp` must return the largest finite double rather than infinity, the tiny-input result moves off 1, and the underflow boundary moves. Keep one table per mode, or one table with a column per mode.

Special cases are often where an implementation first reveals whether it was designed from the contract or from the ordinary finite path. The finite path is only one part of the function.

25 Rounding modes and dispatch policy

A library function should match the C++ math contract for the function being implemented. That includes finite values, special cases, overload behavior, return type, NaNs, infinities, signed zeros, subnormals, overflow, underflow, and rounding modes.

For floating-point math functions, a strong target is correct behavior under all four IEEE rounding modes from Section 22.

That affects dispatch.

A compiler builtin or platform libm call should not be assumed correct outside round-to-nearest. If a runtime path uses a builtin, SIMD implementation, or platform function that is only known safe under `FE_TONEAREST`, the dispatcher must check the live rounding mode first.

Directed modes should route to the generic kernel unless that fast path has been proved and tested for the active mode.

The public header should not bypass that guard at runtime. Constant evaluation may use an eligible `constexpr` builtin when it satisfies the same edge-case and accuracy policy. Runtime calls should go through the runtime dispatcher so the live rounding mode is observed.

That is part of the function contract. A polynomial that works under round-to-nearest is not automatically correct under `FE_UPWARD`, `FE_DOWNWARD`, or `FE_TOWARDZERO`. The final rounding step, signed-zero cases, overflow thresholds, and underflow behavior may depend on the active mode.

26 Tool roles

26.1 Sollya

Sollya bounds approximation error for a stated function, interval, error model, and coefficient format. That step does not cover reduction, evaluation, reconstruction, or final rounding.

26.2 Gappa

Gappa handles floating-point error bounds in straight-line code [7].

This fits Horner kernels, Estrin kernels, compensated sums, split constants, FMA sequences, and reconstruction fragments where the operation order is fixed.

Sollya bounds the mathematical approximation error. Gappa helps bound the floating-point evaluation error. A worked account of certifying a real elementary function this way is given in [8].

26.3 Rocq

Rocq is an interactive theorem prover.

Reach for it when the proof needs a larger machine-checked structure: definitions, lemmas, interval facts, rounding facts, and the link between local Gappa proofs and the function-level theorem.

Rocq belongs to the high-assurance path, not the first pass for every function.

26.4 MPFR

MPFR is the independent oracle [11].

MPFR testing is not a proof, but it is the usual way to validate an implementation at scale. Test random inputs, boundary inputs, subnormals, overflow and underflow thresholds, exact cases, hard reduction cases, hard rounding cases, and every supported rounding mode.

Measure ULP error. Do not rely on printed decimal comparisons. The harness from the sine example, with its order-preserving bit mapping, is the reusable core of such a suite.

Function	Typical reduction
$\exp(x)$	2^k times $\exp(r)$
$\log(x)$	mantissa and exponent split
trig	reduction modulo $\pi/2$ or $\pi/4$
$\mathrm{pow}(x, y)$	special integer paths, logarithmic paths, exact cases

26.5 SageMath

SageMath is useful for symbolic manipulation, series expansion, exact rational arithmetic, continued fractions, interval experiments, and prototype derivations.

For example, before writing a `log1p` kernel, inspect:

$$\frac{\log(1+x)}{x}$$

Before writing a trig reducer, inspect continued-fraction approximants of $\pi/2$.

SageMath is a scratch tool. Sollya is the production coefficient path.

27 Checklist for a new function

27.1 Specify the target

Write down the function, supported formats, supported rounding modes, domain, special cases, accuracy target, and whether the goal is correct rounding, faithful rounding, or a stated ULP bound.

Do this before writing coefficients or special-case code.

27.2 Choose the reduction

Decide how the input maps into a small interval.

Also decide how much precision the reduction carries.

Record the sign, exponent, and significand split when the function uses it.

27.3 Define the kernel

Write the exact function that Sollya will approximate.

This may be $\sin(r)$, $\sin(r)/r$, $\exp(r)$, $\mathrm{expm1}(r)/r$, $\log(1+r)/r$, a table residual, or another transformed function.

27.4 Choose the interval

Write the interval explicitly.

$$r \in \left[-\frac{\log 2}{2}, \frac{\log 2}{2}\right]$$

$$r \in \left[-\frac{\pi}{4}, \frac{\pi}{4}\right]$$

An approximation proof only holds on the interval that was checked.

27.5 Choose the error model

Choose absolute error, relative error, or a weighted error.

Prefer relative error when it matches ULP behavior. Switch to absolute error or a transformed kernel when relative error is ill-conditioned.

27.6 Generate coefficients

`guessdegree` gives a first degree estimate. Test nearby degrees.

`fpminimax` is the production coefficient tool.

```
prec = 200!;
display = hexadecimal!;
```

Generate coefficients in the same formats used by the implementation.

27.7 Generate the tables

For each lookup table, record the invariant.

At minimum, write down the index formula, the represented real value, the split format, the exactness constraint, the residual interval, and the precision tier that consumes the table.

Generate separate tables for separate precision tiers when the split or exactness constraint changes.

For log-style tables, record the residual formula, such as $u = r_i(1 + m_x) - 1$, and the correction term supplied by the table.

27.8 Check the approximation

`dirtyinfnorm` is fine for quick iteration. `supnorm` is the recorded bound.

Record the interval, polynomial degree, coefficient formats, error model, and bound.

27.9 Choose the evaluation scheme

Choose Horner, Estrin, compensated Horner, double-word evaluation, or a hybrid.

Treat the evaluation choice as part of the proof.

27.10 Choose the precision tiers

Decide which inputs use the fast path and which inputs need an accurate path.

For float, double or double-double may be enough for many fast paths.

For double, expect a double-double-like fast path and a wider accurate path for hard cases.

Use dyadic formats when the accurate path needs fixed precision and wider exponent range.

27.11 Prove the evaluation error

Bound the floating-point error from the actual operation sequence.

Run Gappa where possible. The approximation error alone is not enough.

Keep the total error budget visible. The polynomial bound is only one term.

27.12 Reconstruct and round

Undo the reduction. Handle scaling, quadrant selection, sign reconstruction, table lookup, compensated addition, overflow, underflow, and double-rounding hazards where they apply.

If the target is correct rounding, handle hard-to-round inputs or provide a bounded strategy that covers them.

27.13 Validate

Compare against MPFR over random, structured, boundary, subnormal, overflow, underflow, cancellation-heavy, and adversarial inputs.

Run every supported rounding mode.

27.14 Record the status

Document supported formats, supported rounding modes, maximum observed ULP, proven bound if one exists, unsupported cases, known gaps, test coverage, oracle used, and the relevant Sollya, Gappa, SageMath, or Rocq artifacts.

28 Where this lives in a library

Most implementations end up with the same pieces: special-case handling, reduction constants, approximation coefficients, an evaluation kernel, reconstruction logic, rounding-mode handling, oracle tests, and status notes.

Generated constants are part of the algorithm. If the interval, coefficient format, polynomial degree, table split, or evaluation order changes, the proof needs review.

Keep public APIs clean. Put function-specific machinery in internal support code. Keep kernels small enough to audit. Store or document enough information to regenerate the constants.

If a function is only faithful, say so. If only round-to-nearest has been validated, say so. If directed modes are not covered, say so.

Do not imply a stronger contract than the proof and tests support.

29 Other approaches

29.1 Traditional libm workflow

In practice, the workflow matches Figure 17, with explicit bounds on floating-point evaluation error before the final rounding argument. Sollya and Gappa fit this structure naturally.

29.2 RLIBM-style result approximation

RLIBM uses a different target [22].

The traditional route approximates the real function, then proves that rounding gives the desired floating-point result.

RLIBM generates polynomials that target the correctly rounded floating-point result directly. The polynomial is trying to land inside the interval of real values that round to the right floating-point answer.

Not the default workflow in this guide. Worth studying for correctly rounded float functions, generated libraries, and multi-format work.

29.3 CRlibm and CORE-MATH

CRlibm is useful for studying Ziv-test based correctly rounded functions [5, 6].

CORE-MATH is useful for modern correctly rounded elementary functions with compact implementations and proof-oriented notes [5].

Study the structure: reduction, approximation, reconstruction, hard cases, proof.

30 Minimal Sollya template

Start from the script in Section 10. For a reproducible production script, also print the certified bound and the coefficient forms:

```
eps_cert = supnorm(p, f, I, relative, 1b-40);
print("certified relative error:", eps_cert);

print("certified relative error bits:", log2(eps_cert));

print("canonical:");
canonical(p);

print("horner:");
horner(p);
```

Keep the script with the implementation or proof tooling. Constants in source should be reproducible from the script.

31 Minimal SageMath scratch template

Use SageMath before the production Sollya script when the algebra is still being explored.

```
var('x')

f = log(1 + x) / x
series(f, x, 0, 10)
```

For continued fractions:

```
continued_fraction(pi / 2).convergents()[:10]
```

For exact rational checks:

```
R = RationalField()
a = R(355) / R(113)
a - pi.n(100)
```

Treat these as exploration unless the exact claim is recorded and checked as part of the proof.

Source	Typical use in this guide
IEEE 754-2019 [14]	Required operations, rounding modes, exceptions, NaNs, infinities, signed zeros, subnormals
Muller, *Elementary Functions* [17]	Minimax approximation, Remez, range reduction, Cody-Waite, Payne-Hanek, Ziv tests
Handbook of Floating-Point Arithmetic [18]	ULPs, FMA, double-word arithmetic, compiler hazards, Gappa, MPFR
Sollya [25]	Command syntax and coefficient generation
Gappa [12]	Straight-line floating-point error proofs
de Dinechin, Lauter, Muller [9]	Production table-based log with correct-rounding proof
de Dinechin, Lauter, Melquiond [8]	Certifying an elementary function with Gappa
Ziv [29], Payne and Hanek [21], Lefèvre and Muller [16]	Original papers for the named algorithms and worst-case searches
Brisebarre and Chevillard [3]	The method behind `fpminimax`
Tool papers [4, 7, 11]	Citable descriptions of Sollya, MPFR, and Gappa
Rocq [27]	Machine-checked proof structure
SageMath [23]	Symbolic exploration before production scripts
MPFR [26]	Oracle reference behavior
RLIBM [22]	Direct approximation of the correctly rounded result
Goldberg [13], Oracle NCG [20]	Introductory floating-point background

32 Measuring performance

Speed numbers are as easy to get wrong as accuracy numbers, and wrong in quieter ways.

Benchmark the function the way applications call it. Feed inputs from a prefilled array rather than a constant, so the compiler cannot fold the call away and the branch predictor cannot memorize one special-case path. Consume every result, a summed sink variable is enough. Measure latency and throughput separately and label which is which: latency chains each call's result into the next call's input, while throughput runs independent calls. On kernels that vectorize well the two can differ by several times.

Input distribution is part of the result. A benchmark of `exp` over $[0, 1]$ never visits the subnormal, overflow, or accurate-path branches, and a benchmark of `sin` over $[10^{15}, 10^{16}]$ does nothing but large-argument reduction. Report the range with every number, and benchmark the slow paths separately so a regression in them is visible at all.

Compare against the platform libm compiled with the same flags, on the same input set, in the same harness. Numbers taken from different harnesses are not comparable. Once the algorithm is fixed, instruction-level profilers and hardware counters help with the last factors, but the biggest levers are design choices: polynomial degree, table size against cache pressure, and how rarely the accurate path triggers.

33 Citation policy

Citations use ACM numeric style with `ACM-Reference-Format.bst`. Prefer full author names and stable DOI or URL fields in the bibliography.

34 Further reading

The bibliography lists the works cited in the body. The notes below give a reading path instead: free resources first, then paid books for deeper derivations. Each entry lists a URL and what it covers. Where an entry has a citable paper behind it, the bibliography entry is the stable reference.

34.1 Free resources

34.1.1 First pass on floating point. Oracle reprint of Goldberg, What Every Computer Scientist Should Know About Floating-Point Arithmetic. https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

Introductory floating-point article. Covers rounding, guard digits, cancellation, ULPs, and IEEE arithmetic.

The Floating-Point Guide. https://floating-point-gui.de/

Short beginner reference on binary floating-point behavior, not libm design.

IEEE 754-2019 official page. https://standards.ieee.org/standard/754-2019.html

Official standard page. The full text is not free on every access path.

34.1.2 Sollya and approximation work. Sollya official site. https://www.sollya.org/

Tool site and current manual. Minimax approximation, constrained coefficients, and certified supremum-norm checks.

Sollya `fpminimax` documentation. https://www.sollya.org/sollya-current/help.php?name=fpminimax

Command reference for production coefficients in requested floating-point formats.

Sollya `supnorm` documentation. https://www.sollya.org/sollya-current/help.php?name=supnorm

Recorded approximation bounds. Run after quick `dirtyinfnorm` checks.

34.1.3 Proof and validation tools. Gappa user guide. https://gappa.gitlabpages.inria.fr/gappa/index.html

Straight-line floating-point and fixed-point error proofs after Sollya.

Rocq Prover reference manual. https://rocq-prover.org/refman

Machine-checked proof structure beyond local Gappa bounds.

GNU MPFR official site. https://www.mpfr.org/

High-precision oracle with correct rounding.

SageMath documentation. https://doc.sagemath.org/

Symbolic exploration before production Sollya scripts: series, exact rationals, and continued fractions.

34.1.4 Implementations and current research. LLVM libc math status. https://libc.llvm.org/math/

LLVM libc math accuracy table, including functions marked correctly rounded in all four rounding modes.

CORE-MATH project. https://core-math.gitlabpages.inria.fr/

Compact correctly rounded implementations and proof-oriented notes.

CORE-MATH paper. https://inria.hal.science/hal-03721525v3/file/core-math-final.pdf

Project goals and design direction.

RLIBM project page. https://people.cs.rutgers.edu/~sn349/rlibm/

Polynomials that target the correctly rounded floating-point result directly.

RLIBM-ALL paper. https://arxiv.org/abs/2108.06756

Multi-format, multi-rounding-mode RLIBM.

RLIBM-MultiRound paper. https://arxiv.org/abs/2504.07409

Rounding-mode switching cost and methods that avoid depending on the live mode.

CRlibm report. https://ens-lyon.hal.science/ensl-01529804v1/document
Correctly rounded elementary functions and Ziv-style design.

Fast and correctly rounded logarithms in double-precision. https://ens-lyon.hal.science/ensl-00000007v2
Production-grade version of the table-based log core built in Section 13, with the full proof structure [9].

Certifying the floating-point implementation of an elementary function using Gappa. https://arxiv.org/abs/0801.0523
End-to-end account of proving a real kernel's evaluation error [8].

SIGPLAN blog introduction to RLIBM. https://blog.sigplan.org/2021/08/26/high-performance-correctly-rounded-math-libraries-for-32-bit-floating-point-representations/
Readable introduction to the RLIBM approach without the paper formalism [19].

34.1.5 *Performance-oriented implementation references. SLEEF paper.* https://www.shibatch.net/sleef/
Portable vectorized elementary functions with explicit vector paths and accuracy targets [24].

LLVM libc math slides. https://libc.llvm.org/math/
Modern CPU instructions, FMA, and rounding instructions in LLVM libc math work.

Arm Optimized Routines. https://www.arm.com/why-arm/technologies/compute-library
Target-specific scalar and vector math routines for Arm CPUs [2].

Agner Fog optimization manuals. https://www.agner.org/optimize/
CPU microarchitecture, instruction timing, and low-level performance notes [10].

Intel optimization manual. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
Intel x86 tuning and instruction behavior [15].

AMD Zen optimization guide. https://developer.amd.com/resources/developer-guides-manuals/
AMD-specific tuning for Zen-family processors [1].

34.1.6 *Survey and background papers. Floating-point arithmetic, Acta Numerica survey.* https://www.cambridge.org/core/journals/acta-numerica/article/floating-point-arithmetic/287C4D5F6D4A43FBEEB1ABED2A405AAF
Modern survey of floating-point arithmetic and IEEE behavior.

Gappa paper, Certification of bounds on expressions involving rounded operators. https://arxiv.org/abs/cs/0701186
Paper behind the Gappa tool.

MPFR paper, A Multiple-Precision Binary Floating-Point Library with Correct Rounding. https://inria.hal.science/inria-00070266v1/document
Design of the usual high-precision oracle.

34.2 Paid books and standards

34.2.1 *Best main book for elementary functions. Elementary Functions: Algorithms and Implementation*, Jean-Michel Muller, Springer. https://link.springer.com/book/10.1007/978-1-4899-7983-4
Exp, log, trig functions, range reduction, polynomial approximation, table methods, and correct rounding.

34.2.2 *Best main book for floating-point arithmetic. Handbook of Floating-Point Arithmetic*, 2nd edition, Springer. https://link.springer.com/book/10.1007/978-3-319-76526-6
Floating-point formats, rounding, ULPs, FMA, exceptions, compiler issues, Gappa, MPFR, and formal verification.

34.2.3 Strong general book on finite precision error. Accuracy and Stability of Numerical Algorithms, Nicholas J. Higham, SIAM. https://epubs.siam.org/doi/10.1137/1.9780898718027
Numerical error, stability, and finite precision analysis, not libm design.

34.2.4 Shorter IEEE floating-point book. Numerical Computing with IEEE Floating Point Arithmetic, Michael L. Overton, SIAM. https://epubs.siam.org/doi/book/10.1137/1.9780898718072
Shorter IEEE floating-point introduction than the Handbook.

34.2.5 Historical libm implementation reference. Software Manual for the Elementary Functions, Cody and Waite. https://www.amazon.com/Elementary-Functions-Prentice-Hall-computational-mathematics/dp/0138220646
Classic libm structure: reduction, rational approximation, special cases, and implementation constants. Not a modern correctness target.

34.2.6 Official floating-point standard. IEEE 754-2019 official standard page. https://standards.ieee.org/standard/754-2019.html
Authoritative standard for floating-point formats, rounding attributes, exceptions, NaNs, infinities, signed zeros, subnormals, and required operations.

References

[1] Advanced Micro Devices. [n. d.]. Software optimization guides for AMD processors. https://developer.amd.com/resources/developer-guides-manuals/. Accessed 2026-06-08.

[2] Arm Limited. [n. d.]. Arm Optimized Routines. https://www.arm.com/why-arm/technologies/compute-library. Accessed 2026-06-08.

[3] Nicolas Brisebarre and Sylvain Chevillard. 2007. Efficient polynomial L^∞-approximations. In *18th IEEE Symposium on Computer Arithmetic (ARITH-18)*. 169–176.

[4] Sylvain Chevillard, Mioara Joldes, and Christoph Lauter. 2010. Sollya: An environment for the development of numerical codes. In *Mathematical Software – ICMS 2010 (Lecture Notes in Computer Science, Vol. 6327)*. Springer, 28–31.

[5] CORE-MATH Project. [n. d.]. CORE-MATH. https://core-math.gitlabpages.inria.fr/. Accessed 2026-06-08.

[6] Catherine Daramy-Loirat, David Defour, Florent de Dinechin, Matthieu Gallet, Nicolas Gast, and Jean-Michel Muller. 2006. *CR-LIBM: A library of correctly rounded elementary functions in double-precision.* Technical Report. LIP. https://ens-lyon.hal.science/ensl-01529804v1/document Technical report describing version 0.10beta.

[7] Marc Daumas and Guillaume Melquiond. 2010. Certification of bounds on expressions involving rounded operators. *ACM Trans. Math. Software* 37, 1 (2010).

[8] Florent de Dinechin, Christoph Lauter, and Guillaume Melquiond. 2011. Certifying the floating-point implementation of an elementary function using Gappa. *IEEE Trans. Comput.* 60, 2 (2011), 242–253. doi:10.1109/TC.2010.128

[9] Florent de Dinechin, Christoph Lauter, and Jean-Michel Muller. 2007. Fast and correctly rounded logarithms in double-precision. *RAIRO - Theoretical Informatics and Applications* 41, 1 (2007), 85–102. doi:10.1051/ita:2007003

[10] Agner Fog. [n. d.]. Optimization manuals. https://www.agner.org/optimize/. Accessed 2026-06-08.

[11] Laurent Fousse, Guillaume Hanrot, Vincent Lefèvre, Patrick Pélissier, and Paul Zimmermann. 2007. MPFR: A multiple-precision binary floating-point library with correct rounding. *ACM Trans. Math. Software* 33, 2 (2007).

[12] Gappa. [n. d.]. Gappa user guide. https://gappa.gitlabpages.inria.fr/gappa/index.html. Accessed 2026-06-08.

[13] David Goldberg. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *Comput. Surveys* 23, 1 (1991), 5–48. doi:10.1145/103162.103163

[14] IEEE. 2019. IEEE Standard for Floating-Point Arithmetic. doi:10.1109/IEEESTD.2019.8766229

[15] Intel Corporation. [n. d.]. Intel 64 and IA-32 Architectures Software Developer's Manual. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html. Accessed 2026-06-08.

[16] Vincent Lefèvre and Jean-Michel Muller. 2001. Worst cases for correct rounding of the elementary functions in double precision. In *15th IEEE Symposium on Computer Arithmetic (ARITH-15)*. 111–118.

[17] Jean-Michel Muller. 2016. *Elementary Functions: Algorithms and Implementation* (3 ed.). Birkhäuser. doi:10.1007/978-1-4899-7983-4

[18] Jean-Michel Muller, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, and Serge Torres. 2018. *Handbook of Floating-Point Arithmetic* (2 ed.). Birkhäuser. doi:10.1007/978-3-319-76526-6

[19] Santosh Nagarakatte. [n. d.]. High performance correctly rounded math libraries for 32-bit floating point representations. https://blog.sigplan.org/2021/08/26/high-performance-correctly-rounded-math-libraries-for-32-bit-floating-point-representations/. Accessed 2026-06-10.
[20] Oracle. [n. d.]. Numerical Computation Guide. https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html. Accessed 2026-06-08.
[21] Mary H. Payne and Robert N. Hanek. 1983. Radian reduction for trigonometric functions. *ACM SIGNUM Newslett.* 18, 1 (1983), 19–24.
[22] RLibm Project. [n. d.]. Rlibm project page. https://people.cs.rutgers.edu/~sn349/rlibm/. Accessed 2026-06-08.
[23] SageMath Developers. [n. d.]. SageMath documentation. https://doc.sagemath.org/. Accessed 2026-06-08.
[24] Naoki Shibata. 2018. SLEEF: A Portable Vectorized Library of Elementary Functions. In *2018 IEEE 25th International Conference on High Performance Computing Workshops (HiPCW)*. 49–54. doi:10.1109/HiPCW.2018.86345
[25] Sollya. [n. d.]. Sollya. https://www.sollya.org/. Accessed 2026-06-08.
[26] The MPFR Team. [n. d.]. GNU MPFR. https://www.mpfr.org/. Accessed 2026-06-08.
[27] The Rocq Prover Development Team. [n. d.]. The Rocq Prover reference manual. https://rocq-prover.org/refman. Accessed 2026-06-08.
[28] Paul Zimmermann, Vincent Lefèvre, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Guillaume Melquiond, Nathalie Revol, and Serge Torres. [n. d.]. CORE-MATH: Fast and correctly rounded elementary functions. https://inria.hal.science/hal-03721525v3/file/core-math-final.pdf. Accessed 2026-06-08.
[29] Abraham Ziv. 1991. Fast evaluation of elementary mathematical functions with correctly rounded last bit. *ACM Trans. Math. Software* 17, 3 (1991), 410–423.

A Useful vocabulary

B How the proof pieces fit together

Keep these obligations separate when reading or writing a proof note.

(1) **Reduction bound:** distance between the computed reduced argument and the exact reduced argument.
(2) **Approximation bound:** distance between the mathematical polynomial and the kernel on the reduced interval.
(3) **Evaluation bound:** rounding error from the compiled operation sequence that evaluates the polynomial.
(4) **Reconstruction bound:** effect of split constants, exponent additions, sign fixes, and table lookups.
(5) **Rounding argument:** margin to a rounding boundary, or the need for a fallback path.
(6) **Oracle validation:** agreement with an independent reference over many inputs. Engineering evidence, not a proof certificate.

Small approximation error does not fix bad reduction. A split table does not prove the polynomial. MPFR agreement does not certify the floating-point circuit. State which obligation each artifact covers.

Term	Meaning
argument reduction	A mapping from the public input to a smaller interval where approximation is easier.
kernel	The exact function that is actually approximated after reduction.
transformed kernel	A kernel rewritten to remove a zero, cancellation point, or unstable relative error.
reduced interval	The interval on which the approximation bound is claimed.
residual	The small quantity that remains after a table lookup or other reduction step.
reconstruction	The step that combines the reduced result with exponents, table constants, signs, or quadrants.
minimax polynomial	A polynomial that minimizes the worst error on a stated interval.
coefficient format	The floating-point or fixed-width format in which the implementation stores the coefficients.
evaluation scheme	The operation order used to evaluate the polynomial, such as Horner or Estrin.
table invariant	The exact real relation that a table entry is meant to support.
exactness constraint	A requirement that one later operation be exact or tightly bounded because of how a constant was split.
faithful rounding	Returning one of the two floating-point neighbors of the exact result.
correct rounding	Returning the value selected by the active rounding mode from the exact real result.
hard case	An input whose exact result lies unusually close to a rounding boundary.
oracle	An independent high-precision reference used for validation, usually MPFR in this workflow.
fast path	The cheap path intended for ordinary inputs.
accurate path	The slower path used when the fast path cannot justify the final rounding decision.
ULP	The gap between consecutive floating-point values at a given magnitude.